Metode Heuristik dan Metaheuristik

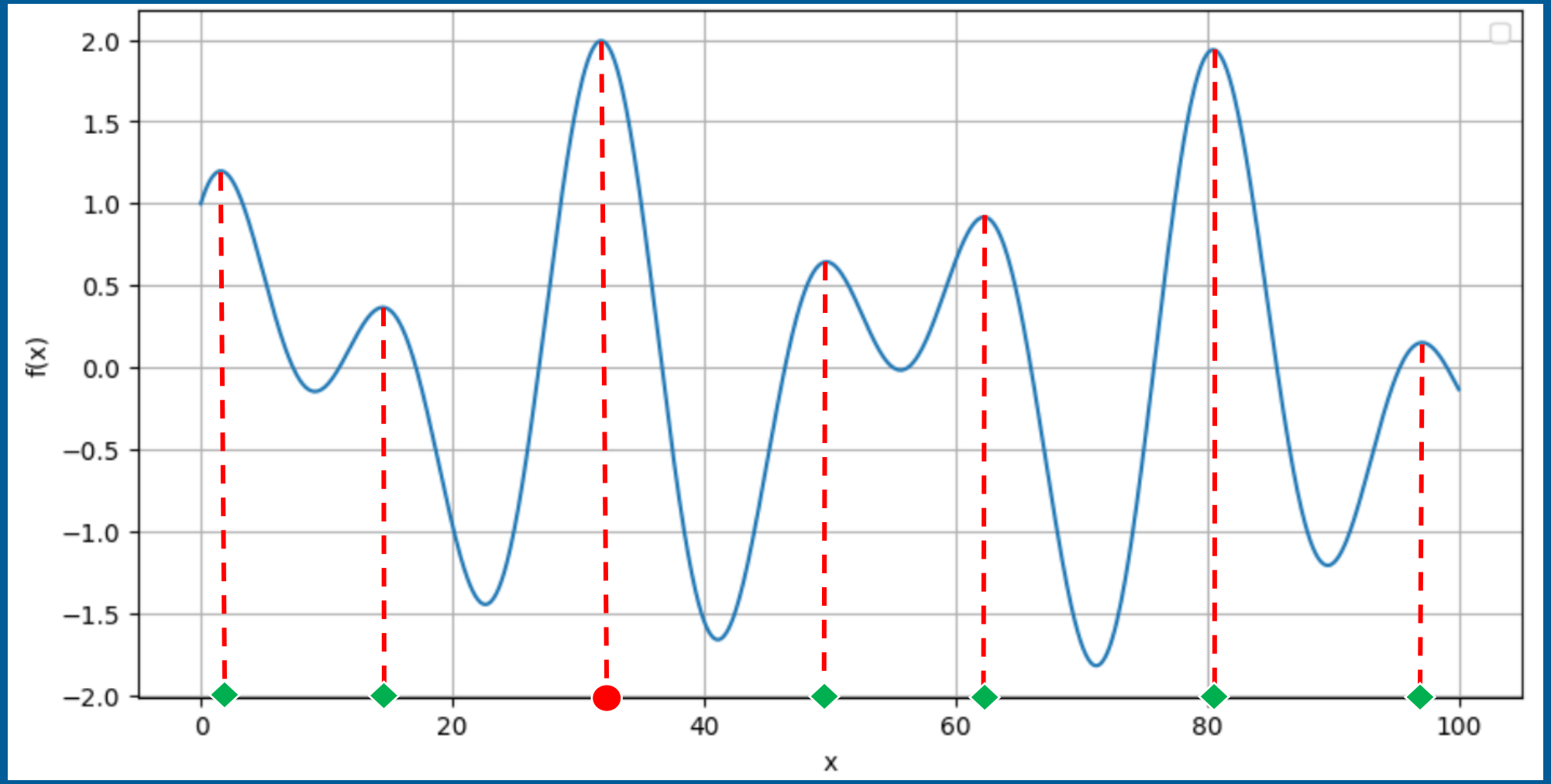
Suprayogi
KK Sistem Industri dan Tekno-Ekonomi
Fakultas Teknologi Industri – Institut Teknologi Bandung
yogi@itb.ac.id

Metode Heuristik

Metode Heuristik

- Pendekatan pencarian solusi yang dirancang untuk menghasilkan solusi yang baik secara cepat, meskipun tidak selalu optimal secara global.
- Solusi yang diperoleh biasanya berupa solusi optimal lokal atau suboptimal, namun cukup layak dan efisien untuk diterapkan dalam praktek, terutama pada masalah dengan ukuran besar atau kompleksitas tinggi.
- Bekerja berdasarkan aturan-aturan pencarian yang logis atau intuitif, tanpa harus melakukan eksplorasi menyeluruh terhadap seluruh ruang solusi seperti pada metode optimal (eksak).
- Waktu komputasi dapat ditekan secara signifikan, meskipun dengan kompromi pada kualitas solusi.

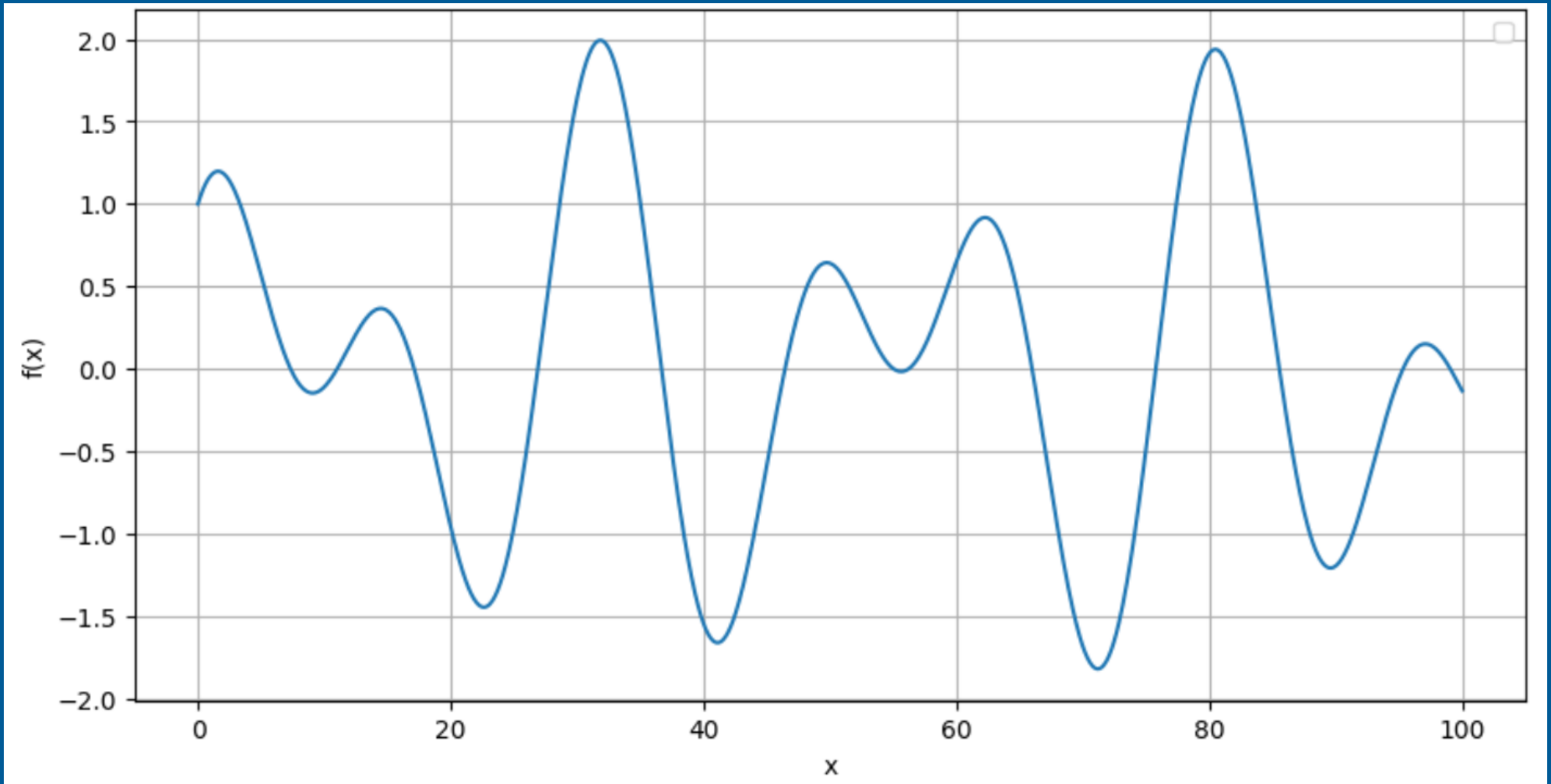
Optimum Global dan Optimum Local



Masalah-Masalah yang Kompleks

- Masalah optimisasi takcembung (nonconvex optimization problem)
- Masalah optimisasi kombinatorial yang sulit (hard combinatorial problem)

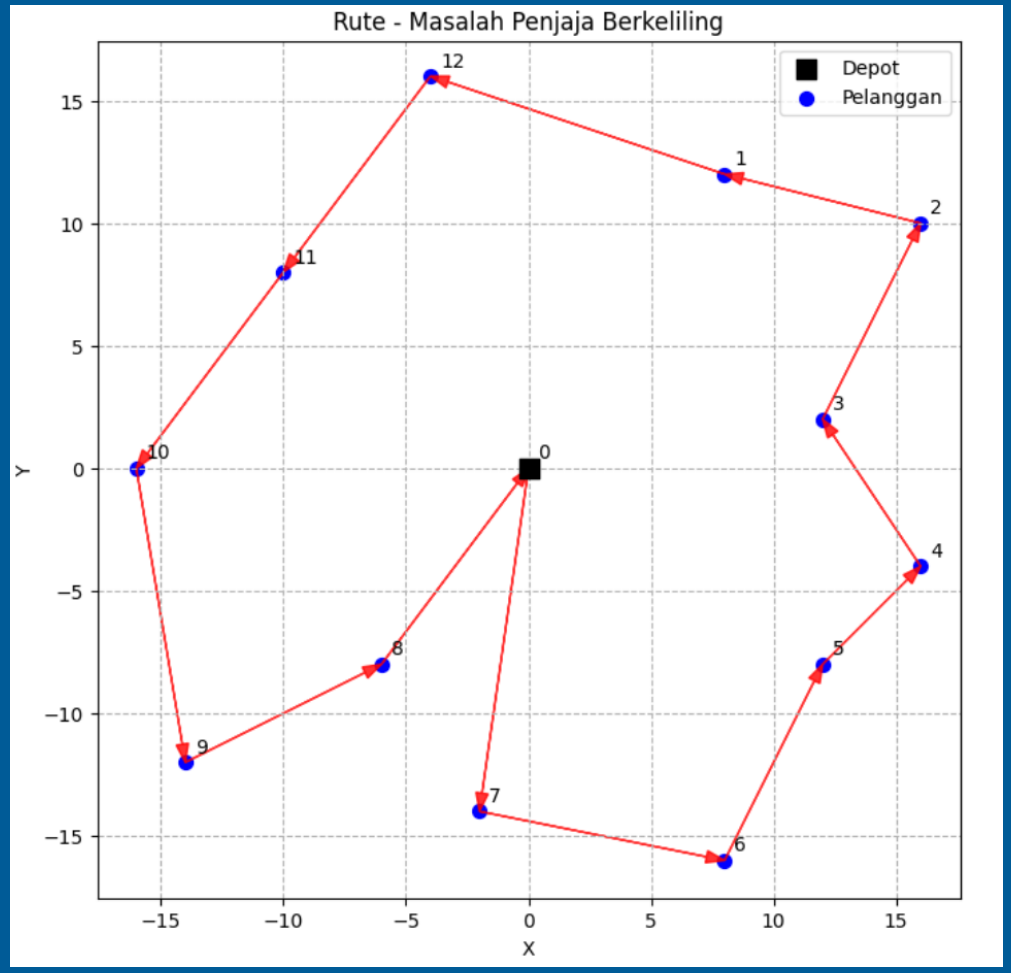
Masalah Optimisasi Takcembung



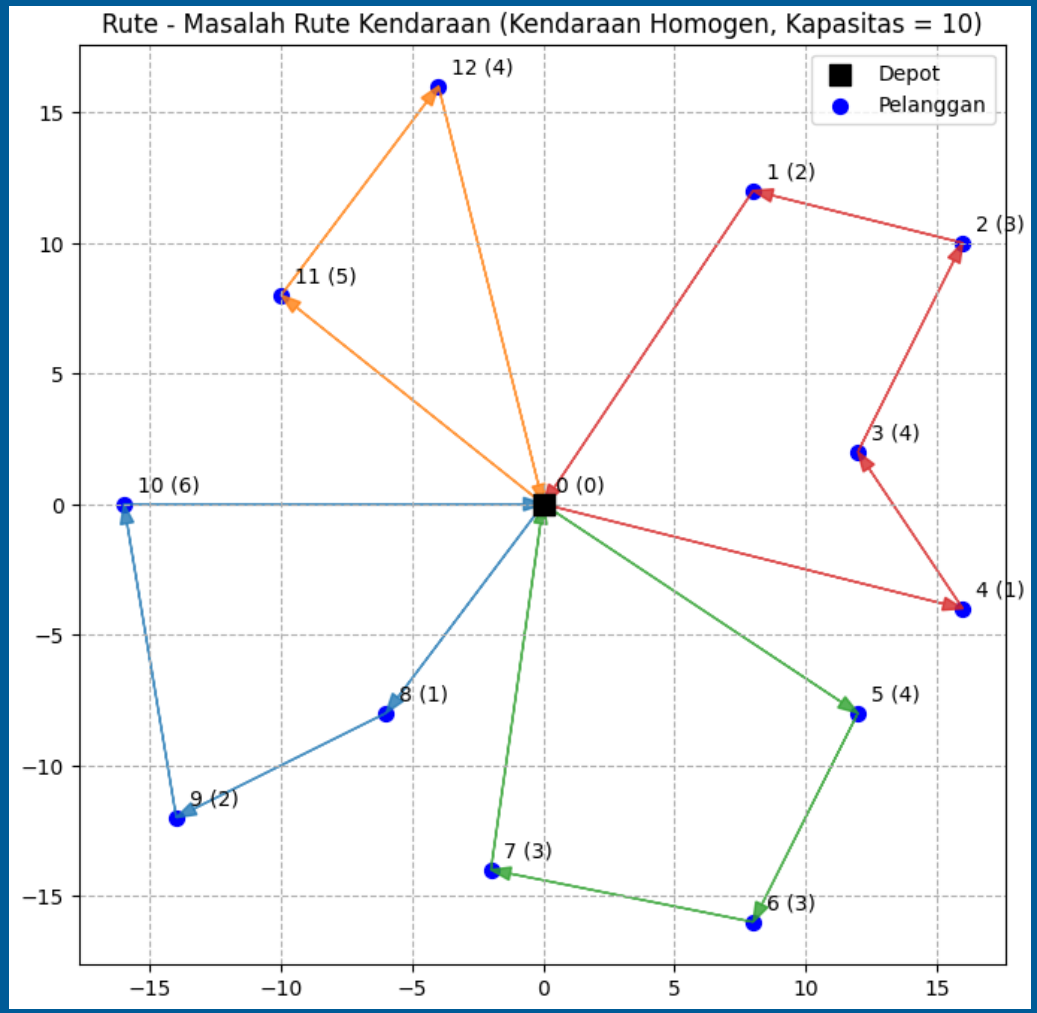
Beberapa Masalah Optimisasi Kombinatorial yang Sulit

- Masalah Penjual Keliling, MPK (Traveling Salesman Problem, TSP)
- Masalah Rute Kendaraan, MRK (Vehicle Routing Problem, VRP)

Masalah Penjual Keliling, MPK (Traveling Salesman Problem, TSP)



Masalah Rute Kendaraan, MRK (Vehicle Routing Problem, VRP)



Kelas Metode Heuristik

- Metode Heuristik Konstruksi
- Metode Heuristik Perbaikan

Metode Heuristik Konstruksi

Metode Heuristik Konstruksi

- Membangun solusi secara bertahap dari keadaan kosong (tanpa solusi) hingga terbentuk solusi lengkap yang layak.
- Pada setiap langkah, elemen solusi baru dipilih berdasarkan kriteria tertentu, misalnya, biaya minimum, jarak terpendek, atau nilai keuntungan tertinggi, hingga seluruh komponen solusi terpenuhi
- Contoh Metode Heuristik untuk Masalah Penjual Keliling:
  - Nearest Neighbor Algorithm

Nearest Neighbor Algorithm

- Prinsip Dasar:
  - Membangun solusi secara bertahap dengan prinsip "pilih tetangga terdekat berikutnya".
  - Proses dimulai dari satu simpul awal yang dipilih, kemudian simpul berikutnya yang memiliki jarak terpendek dari simpul saat ini dipilih sebagai tujuan berikutnya.
  - Langkah ini diulang hingga semua simpul telah dikunjungi, dan akhirnya tur ditutup dengan kembali ke simpul awal.

- Langkah
  - Langkah 1: Inisialisasi solusi dengan simpul awal (simpul depot).
  - Langkah 2: Tetapkan himpunan simpul yang belum dikunjungi.
  - Langkah 3: Identifikasi simpul terakhir pada solusi.
  - Langkah 4: Pilih simpul dalam himpunan simpul yang belum dikunjungi yang memiliki jarak terpendek dengan simpul terakhir.
  - Langkah 5: Tambahkan simpul terpilih pada solusi.
  - Langkah 6: Hapus simpul terpilih pada himpunan simpul yang belum dikunjungi.
  - Langkah 7: Ulangi langkah-langkah 3–6 hingga himpunan simpul yang belum dikunjungi kosong.
  - Langkah 8: Tambahkan simpul awal (simpul depot) pada solusi untuk menutup

Contoh Penerapan untuk Masalah Penjual Keliling

Tabel Jarak

Dari simpul \ Ke simpul	0	1	2	3	4	5	6	7	8	9
0	0	9	14	10	20	11	15	17	8	3
1	8	0	17	4	13	8	5	1	5	14
2	8	17	0	16	10	14	3	8	4	5
3	9	19	1	0	8	4	19	6	14	12
4	10	5	19	10	0	15	12	19	17	7
5	16	19	1	17	2	0	2	3	18	10
6	13	17	3	18	8	18	0	4	17	5
7	8	6	4	16	11	6	17	0	4	11
8	17	15	20	2	2	7	12	8	0	17
9	17	2	6	8	20	10	6	15	4	0

```
Solusi sementara: [0]
Himpunan simpul belum dikunjungi = [1, 2, 3, 4, 5, 6, 7, 8, 9]
Iterasi: 1
  Simpul terakhir pada solusi: 0
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir  -> simpul: 9
  Solusi sementara: [0, 9]
  Himpunan simpul yang belum dikunjungi = [1, 2, 3, 4, 5, 6, 7, 8]
Iterasi: 2
  Simpul terakhir pada solusi: 9
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir  -> simpul: 1
  Solusi sementara: [0, 9, 1]
  Himpunan simpul yang belum dikunjungi = [2, 3, 4, 5, 6, 7, 8]
Iterasi: 3
  Simpul terakhir pada solusi: 1
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir  -> simpul: 7
  Solusi sementara: [0, 9, 1, 7]
  Himpunan simpul yang belum dikunjungi = [2, 3, 4, 5, 6, 8]
Iterasi: 4
  Simpul terakhir pada solusi: 7
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir  -> simpul: 2
  Solusi sementara: [0, 9, 1, 7, 2]
  Himpunan simpul yang belum dikunjungi = [3, 4, 5, 6, 8]
Iterasi: 5
  Simpul terakhir pada solusi: 2
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir  -> simpul: 6
  Solusi sementara: [0, 9, 1, 7, 2, 6]
  Himpunan simpul yang belum dikunjungi = [3, 4, 5, 8]
```

```
Iterasi: 6
  Simpul terakhir pada solusi: 6
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir -> simpul: 4
  Solusi sementara: [0, 9, 1, 7, 2, 6, 4]
  Himpunan simpul yang belum dikunjungi = [3, 5, 8]
Iterasi: 7
  Simpul terakhir pada solusi: 4
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir -> simpul: 3
  Solusi sementara: [0, 9, 1, 7, 2, 6, 4, 3]
  Himpunan simpul yang belum dikunjungi = [5, 8]
Iterasi: 8
  Simpul terakhir pada solusi: 3
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir -> simpul: 5
  Solusi sementara: [0, 9, 1, 7, 2, 6, 4, 3, 5]
  Himpunan simpul yang belum dikunjungi = [8]
Iterasi: 9
  Simpul terakhir pada solusi: 5
  Simpul yang belum dikunjungi dengan jarak terdekat dari simpul terakhir -> simpul: 8
  Solusi sementara: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8]
  Himpunan simpul yang belum dikunjungi = []
Tambahkan simpul 0 pada solusi sementara: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0]
Solusi lengkap: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0]
Jarak total: 70
```

Metode Heuristik Perbaikan

Metode Heuristik Perbaikan

- Pendekatan yang berangkat dari suatu solusi awal yang layak dan berupaya meningkatkan kualitas solusi tersebut melalui proses iteratif.
- Solusi awal ini ditetapkan sebagai solusi terbaik sementara, kemudian dilakukan serangkaian upaya perbaikan dengan membangkitkan solusi-solusi baru yang lebih baik.
- Solusi baru yang dihasilkan disebut sebagai tetangga (neighbor), yaitu solusi lain yang diperoleh dengan melakukan perubahan tertentu (move) pada solusi saat ini.
- Jika solusi baru memiliki nilai fungsi tujuan yang lebih baik, maka solusi tersebut menggantikan solusi sebelumnya dan proses pencarian dilanjutkan hingga tidak ada lagi perbaikan signifikan yang dapat diperoleh atau kriteria penghentian terpenuhi.

Local Search

- Metode heuristik perbaikan yang paling umum digunakan untuk masalah optimisasi kombinatorial

- Prinsip Dasar:
  - Memulai dari suatu solusi awal.
  - Menghasilkan satu atau beberapa solusi tetangga melalui suatu operator atau struktur ketetanggaan.
  - Memilih solusi terbaik di antara tetangga tersebut.
  - Jika solusi baru lebih baik, maka solusi tersebut menjadi solusi saat ini dan proses diulang.
  - Proses berhenti ketika tidak ditemukan lagi tetangga yang memberikan perbaikan terhadap solusi saat ini.

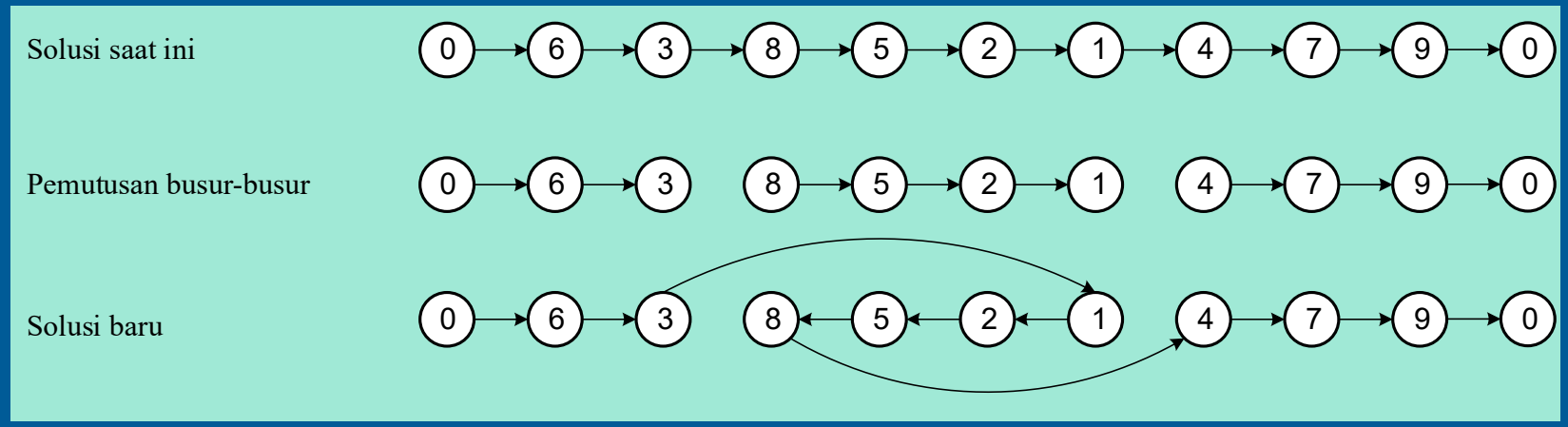
Beberapa Operator (Struktur Ketetanggaan) untuk Masalah Penjual Keliling

- Operator 2-Opt (2-Opt Operator)
- Operator 3-Opt (3-Opt Operator)
- Operator Pertukaran (Swap Operator)
- Operator Relokasi (Relocation Operator)

Operator 2-Opt

- Melibatkan pergerakan dengan memutus dua busur yang tidak berdekatan, membentuk kembali dua busur serta membalik arah beberapa busur.
- Misal busur-busur yang tidak berdekatan yang dihapus pada solusi saat ini adalah $(i, i + 1)$ dan $(j, j + 1)$
- Solusi baru yang dibentuk adalah dengan pembentukan busur-busur (i, j) dan $(i + 1, j + 1)$ serta pembalikan arah-arah busur mulai dari busur $(i + 1, i + 2)$ hingga busur $(j - 1, j)$

Ilustrasi Operator 2-Opt



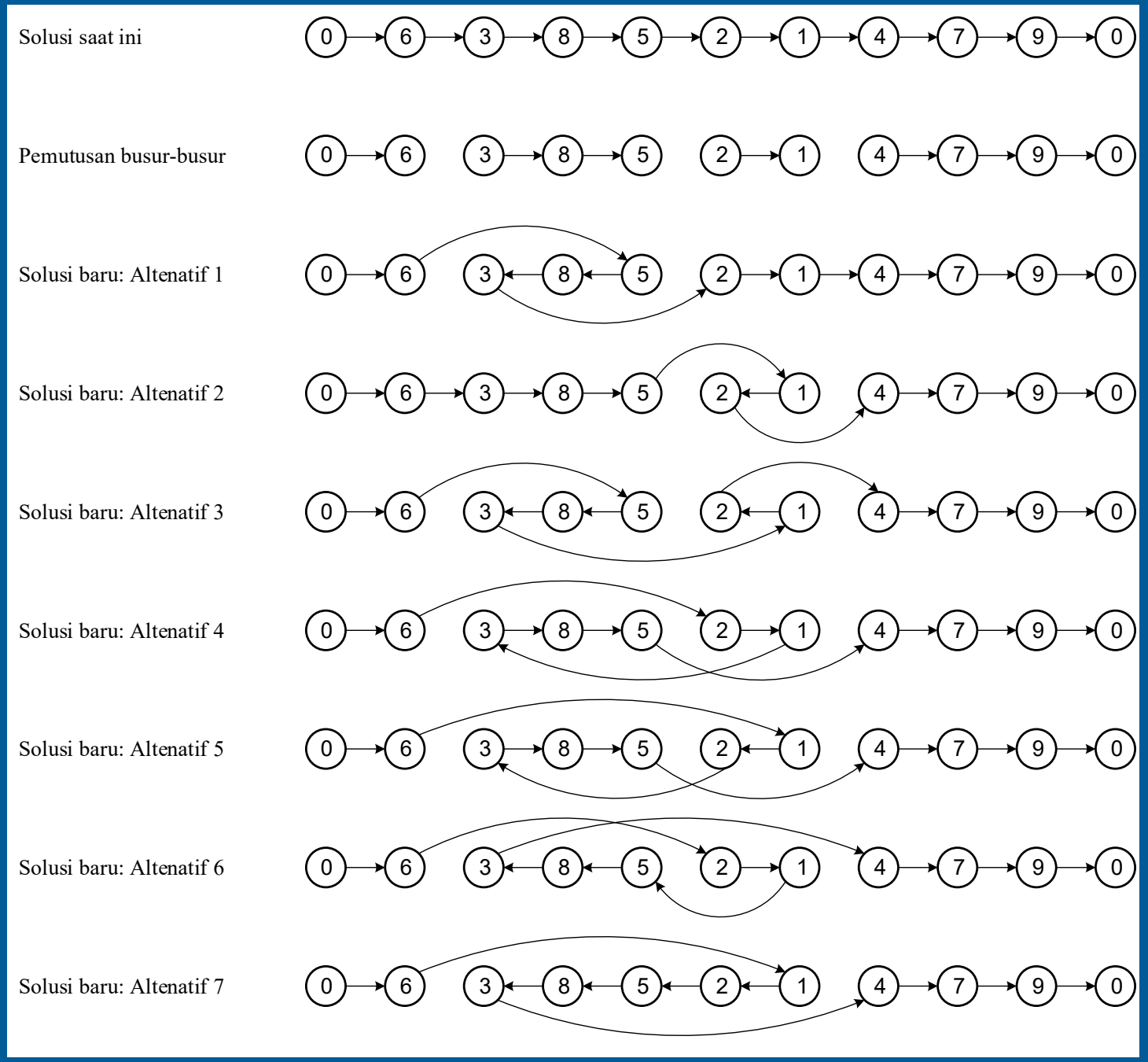

```
Operator 2-opt:
Catatan: Indeks pertama dimulai dari 0

Solusi saat ini: [0, 6, 3, 8, 5, 2, 1, 4, 7, 9, 0]
Proses pergerakan:
 Pilih indeks dari simpul pangkal untuk busur pertama yang diputus: 2
  -> Pasangan indeks dari busur pertama = (2, 3) atau busur simpul = (3, 8)
 Pilih indeks dari simpul pangkal untuk busur kedua yang diputus: 6
  -> Pasangan indeks dari busur kedua = (6, 7) atau busur simpul = (1, 4)
 Identifikasi segmen-segmen urutan simpul yang dihasilkan:
  Segmen 1 (simpul mulai indeks 0 s/d simpul pada indeks 2): [0, 6, 3]
  Segmen 2 (simpul mulai indeks 3 s/d pada indeks 6): [8, 5, 2, 1]
  Segmen 3 (simpul mulai indeks 7 s/d simpul pada indeks 10): [4, 7, 9, 0]
 Pembentukan solusi baru:
  Salin segmen 1: [0, 6, 3]
  Balik dan salin segmen 2: [8, 5, 2, 1] -> [1, 2, 5, 8]
  Salin segmen 3: [4, 7, 9, 0]
  Solusi baru: [0, 6, 3, 1, 2, 5, 8, 4, 7, 9, 0]
```

Operator 3-Opt

- Melibatkan pergerakan dengan memutus tiga busur yang tidak berdekatan sekaligus.
- Selanjutnya proses dilanjutkan dengan membentuk atau menyambungkan kembali busur-busur dan membalik arah beberapa busur.
- Misal busur-busur yang tidak berdekatan yang diputuskan pada solusi saat ini adalah $(i, i + 1)$, $(j, j + 1)$, dan $(i, i + 1)$
- Berdasarkan busur-busur yang diputus ini, terdapat tujuh solusi baru alternatif yang dapat dibentuk melalui proses pembentukan atau penyambungan kembali busur serta pembalikan arah dari beberapa busur

Ilustrasi Operator 3-Opt



```
Operator 3-opt:
Catatan: indeks dimulai dari 0

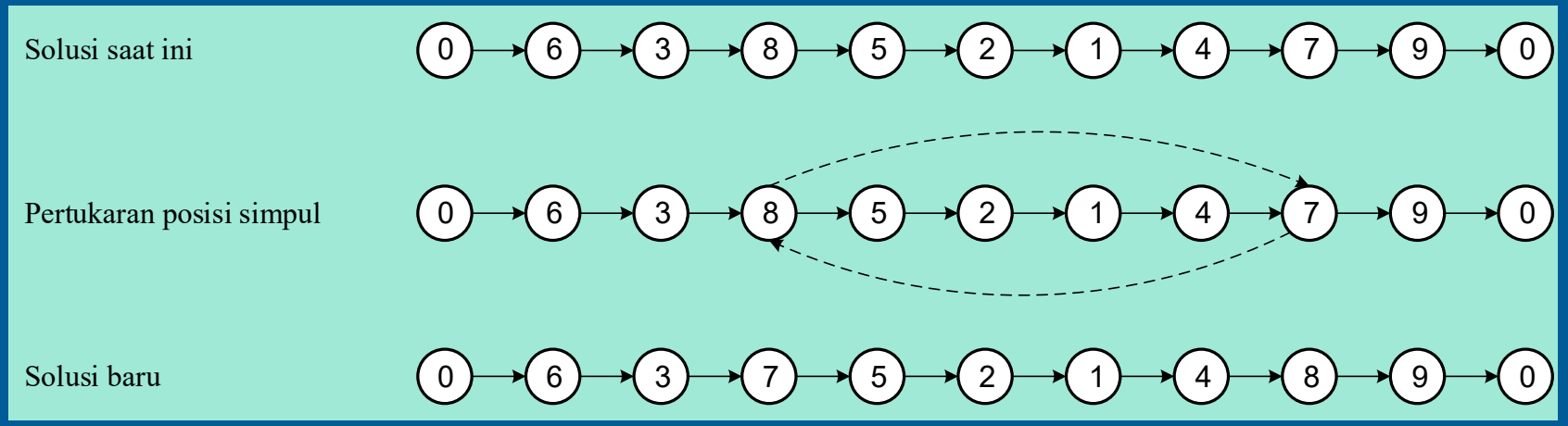
Solusi saat ini: [0, 6, 3, 8, 5, 2, 1, 4, 7, 9, 0]
Proses pergerakan:
 Pilih indeks dari simpul pangkal untuk busur pertama yang diputus: 1
  -> Pasangan indeks dari busur pertama = (1, 2) atau busur simpul = (6, 3)
 Pilih indeks dari simpul pangkal untuk busur kedua yang diputus: 4
  -> Pasangen indeks dari busur kedua = (4, 5) atau busur simpul = (5, 2)
 Pilih indeks dari simpul pangkal untuk busur ketiga yang diputus: 6
  -> Pasangan indeks dari busur ketiga = (6, 7) atau busur simpul = (1, 4)
 Identifikasi segmen-segmen urutan simpul yang dihasilkan:
  Segmen 1 (simpul pada indeks 0 s/d simpul pada indeks 1): [0, 6]
  Segmen 2 (simpul pada indeks 2 s/d simpul pada indeks 4): [3, 8, 5]
  Segmen 3 (simpul pada indeks 5 s/d simpul pada indeks 6): [2, 1]
  Segmen 4 (simpul pada indeks 7 s/d simpul pada indeks 10): [4, 7, 9, 0]
 Pembentukan solusi baru:
  Alternatif 1:
   Salin segmen 1: [0, 6]
   Balik dan salin segmen 2: [3, 8, 5] -> [5, 8, 3]
   Salin segmen 3: [2, 1]
   Salin segmen 4: [4, 7, 9, 0]
   Solusi baru: [0, 6, 5, 8, 3, 2, 1, 4, 7, 9, 0]
  Alternatif 2:
   Salin segmen 1: [0, 6]
   Salin segmen 2: [3, 8, 5]
   Balik dan salin segmen 3: [2, 1] -> [1, 2]
   Salin segmen 4: [4, 7, 9, 0]
   Solusi baru: [0, 6, 3, 8, 5, 1, 2, 4, 7, 9, 0]
```

```
Alternatif 3:
  Salin segmen 1: [0, 6]
  Balik dan salin segmen 2: [3, 8, 5] -> [5, 8, 3]
  Balik dan salin segmen 3: [2, 1] -> [1, 2]
  Salin segmen 4: [4, 7, 9, 0]
  Solusi baru: [0, 6, 5, 8, 3, 1, 2, 4, 7, 9, 0]
 Alternatif 4:
  Salin segmen 1: [0, 6]
  Pindahkan segmen 3 ke posisi segmen 2: [2, 1]
  Pindahkan segmen 2 ke posisi segmen 3: [3, 8, 5]
  Salin segmen 4: [4, 7, 9, 0]
  Solusi baru: [0, 6, 2, 1, 3, 8, 5, 4, 7, 9, 0]
 Alternatif 5:
  Salin segmen 1: [0, 6]
  Balik segmen 3 dan pindahkan ke posisi segmen 2: [2, 1] -> [1, 2]
  Pindahkan segmen 2 ke posisi segmen 3: [3, 8, 5]
  Salin segmen 4: [4, 7, 9, 0]
  Solusi baru: [0, 6, 1, 2, 3, 8, 5, 4, 7, 9, 0]
 Alternatif 6:
  Salin segmen 1: [0, 6]
  Pindahkan segmen 3 ke posisi segmen 2: [2, 1]
  Balik segmen 2 dan pindahkan ke posisi segmen 3: [3, 8, 5] -> [5, 8, 3]
  Salin segmen 4: [4, 7, 9, 0]
  Solusi baru: [0, 6, 2, 1, 5, 8, 3, 4, 7, 9, 0]
 Alternatif 7:
  Salin segmen 1: [0, 6]
  Balik segmen 3 dan pindahkan ke posisi segmen 2: [2, 1] -> [1, 2]
  Balik segmen 2 dan pindahkan ke posisi segmen 3: [3, 8, 5] -> [5, 8, 3]
  Salin segmen 4: [4, 7, 9, 0]
  Solusi baru: [0, 6, 1, 2, 5, 8, 3, 4, 7, 9, 0]
```

Operator Pertukaran

- Bekerja dengan menukar dua simpul yang masing-masing berada pada dua posisi yang berbeda.
- Pertukaran melibatkan semua simpul kecuali simpul depot (simpul 0).

Ilustrasi
Operator Pertukaran



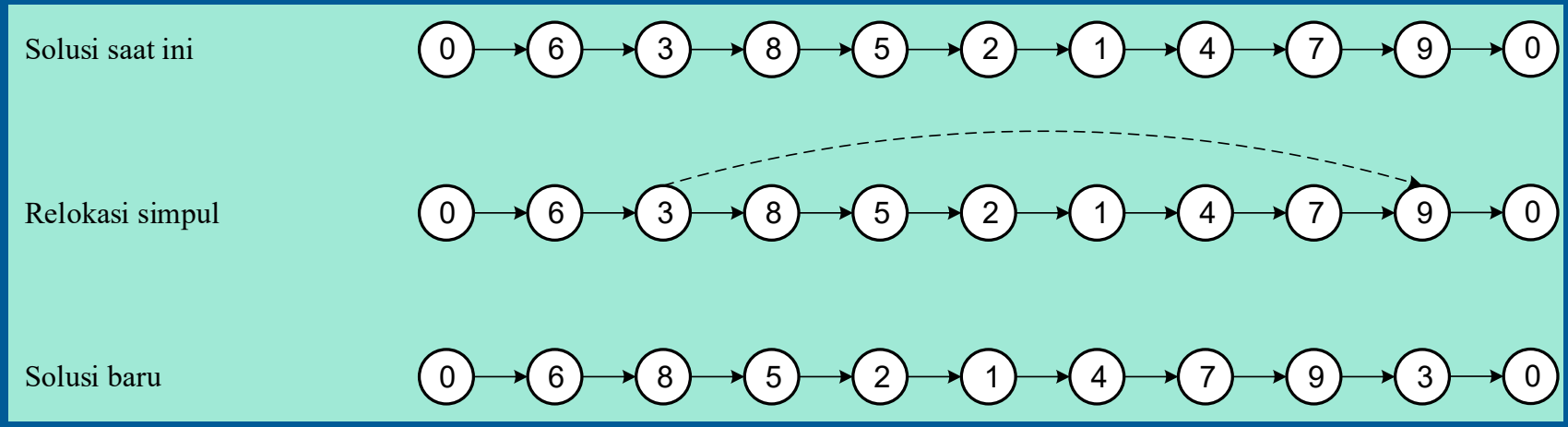

```
Operator Pertukaran
Catatan: indeks dimulai dari 0

Solusi saat ini: [0, 6, 3, 8, 5, 2, 1, 4, 7, 9, 0]
Proses pergerakan:
 Pilih indeks dari simpul pertama yang dipertukarkan:
 -> indeks = 3 (simpul = 8)
 Pilih indeks dari simpul kedua yang dipertukarkan:
 -> indeks = 8 (simpul = 7)
 Pertukarkan indeks
 -> simpul = 8 ke indeks = 8
 -> simpul = 7 ke indeks = 3
 Solusi baru: [0, 6, 3, 7, 5, 2, 1, 4, 8, 9, 0]
```

Operator Relokasi

- Bekerja dengan memindahkan atau merelokasi simpul pada suatu suatu posisi ke posisi yang lain.
- Relokasi ini menyebabkan posisi simpul-simpul sebelum atau sesudahnya menjadi bergeser.
- Relokasi tidak melibatkan simpul depot (simpul 0).

Ilustrasi Operator Relokasi




```
Operator Relokasi
Catatan: indeks dimulai dari 0

Solusi saat ini: [0, 6, 3, 8, 5, 2, 1, 4, 7, 9, 0]
Proses:
 Pilih indeks dari simpul yang akan direlokasi: indeks = 2 (simpul = 3)
 Pilih indeks dari simpul tujuan relokasi: indeks = 9 (simpul = 9)
 Hapus simpul = 3 pada solusi saat ini
  Solusi hasil penghapusan -> [0, 6, 8, 5, 2, 1, 4, 7, 9, 3, 0]
 Sisipkan kembali simpul = 3 pada indeks = 9 dari solusi hasil penghapusan
 Solusi baru:    [0, 6, 8, 5, 2, 1, 4, 7, 9, 3, 0]
```

Strategi Eksplorasi dalam Local Search

- Proses eksplorasi ruang solusi dilakukan dengan membangkitkan sejumlah solusi tetangga dari solusi saat ini.
- Strategi untuk menentukan solusi mana yang akan diterima sebagai solusi baru:
  - Strategi Perbaikan Terbaik (Best Improvement Strategy)
  - Strategi Perbaikan Pertama (First Improvement Strategy)

Strategi Perbaikan Terbaik

- Seluruh solusi tetangga dari solusi saat ini dievaluasi terlebih dahulu.
- Dari semua tetangga yang terbentuk, dipilih solusi dengan nilai fungsi tujuan terbaik (misalnya, jarak total minimum pada Masalah Penjual Keliling) untuk menggantikan solusi saat ini.
- Kelebihan:
  - Cenderung memberikan arah pencarian yang lebih stabil dan konsisten, karena selalu memilih solusi dengan perbaikan maksimum pada setiap iterasi.
- Kelemahan:
  - Biaya komputasi yang lebih tinggi, karena semua tetangga harus dievaluasi sebelum keputusan diambil.

Strategi Perbaikan Pertama

- Solusi tetangga dievaluasi secara berurutan, dan perbaikan pertama yang ditemukan langsung diterima sebagai solusi baru.
- Proses pencarian kemudian dilanjutkan dari solusi tersebut tanpa menunggu evaluasi seluruh tetangga.
- Kelebihan:
  - Umumnya lebih cepat secara komputasi karena tidak memerlukan evaluasi lengkap seluruh tetangga.
- Kelemahan:
  - Hasil akhirnya bisa bergantung pada urutan evaluasi tetangga, sehingga kemungkinan mencapai solusi lokal bisa lebih tinggi.

Contoh Penerapan Local Search untuk Pemecahan Masalah Penjual Keliling

Contoh Penerapan

Tabel Jarak

Dari simpul \ Ke simpul	0	1	2	3	4	5	6	7	8	9
0	0	9	14	10	20	11	15	17	8	3
1	8	0	17	4	13	8	5	1	5	14
2	8	17	0	16	10	14	3	8	4	5
3	9	19	1	0	8	4	19	6	14	12
4	10	5	19	10	0	15	12	19	17	7
5	16	19	1	17	2	0	2	3	18	10
6	13	17	3	18	8	18	0	4	17	5
7	8	6	4	16	11	6	17	0	4	11
8	17	15	20	2	2	7	12	8	0	17
9	17	2	6	8	20	10	6	15	4	0

Ilustrasi Local Search untuk Masalah Penjual Keliling dengan Operator 2-opt dan Strategi Perbaikan Terbaik


```
Metode: Pencarian Lokal (Local Search)
Operator: 2-Opt
Strategi: Perbaikan Terbaik
Metode Penentuan Solusi Awal: Tetangga Terdekat (Nearest Neighbor)

Solusi awal: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0]; Jarak total = 70

Iterasi 1:
Solusi terbaik saat ini: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0]; Jarak total = 70
Pergerakan - Indeks-indeks: (0, 2); Busur-busur yang dihapus: (0, 9), (1, 7);
-> Solusi: [0, 1, 9, 7, 2, 6, 4, 3, 5, 8, 0]; Jarak total = 102
Pergerakan - Indeks-indeks: (0, 3); Busur-busur yang dihapus: (0, 9), (7, 2);
-> Solusi: [0, 7, 1, 9, 2, 6, 4, 3, 5, 8, 0]; Jarak total = 103
...
...
...
Pergerakan - Indeks-indeks: (7, 9); Busur-busur yang dihapus: (3, 5), (8, 0);
-> Solusi: [0, 9, 1, 7, 2, 6, 4, 3, 8, 5, 0]; Jarak total = 68
Semua Pergerakan sudah dieksplorasi. Perbaikan ditemukan.
Solusi terbaik ditemukan: [0, 9, 1, 7, 8, 5, 3, 4, 6, 2, 0]; Jarak total = 66
Eksplorasi mulai kembali dari solusi terbaik saat ini.
```

```
Iterasi 2:
Solusi terbaik saat ini: [0, 9, 1, 7, 8, 5, 3, 4, 6, 2, 0]; Jarak total = 66
Pergerakan - Indeks-indeks: (0, 2); Busur-busur yang dihapus: (0, 9), (1, 7);
-> Solusi: [0, 1, 9, 7, 8, 5, 3, 4, 6, 2, 0]; Jarak total = 98
Pergerakan - Indeks-indeks: (0, 3); Busur-busur yang dihapus: (0, 9), (7, 8);
-> Solusi: [0, 7, 1, 9, 8, 5, 3, 4, 6, 2, 0]; Jarak total = 97
...
...
...
Pergerakan - Indeks-indeks: (7, 9); Busur-busur yang dihapus: (4, 6), (2, 0);
-> Solusi: [0, 9, 1, 7, 8, 5, 3, 4, 2, 6, 0]; Jarak total = 77
Semua Pergerakan sudah dieksplorasi. Perbaikan ditemukan.
Solusi terbaik ditemukan: [0, 9, 1, 7, 8, 4, 3, 5, 6, 2, 0]; Jarak total = 40
Eksplorasi mulai kembali dari solusi terbaik saat ini.

Iterasi 3:
Solusi terbaik saat ini: [0, 9, 1, 7, 8, 4, 3, 5, 6, 2, 0]; Jarak total = 40
Pergerakan - Indeks-indeks: (0, 2); Busur-busur yang dihapus: (0, 9), (1, 7);
-> Solusi: [0, 1, 9, 7, 8, 4, 3, 5, 6, 2, 0]; Jarak total = 72
Pergerakan - Indeks-indeks: (0, 3); Busur-busur yang dihapus: (0, 9), (7, 8);
-> Solusi: [0, 7, 1, 9, 8, 4, 3, 5, 6, 2, 0]; Jarak total = 71
...
...
...
Pergerakan - Indeks-indeks: (7, 9); Busur-busur yang dihapus: (5, 6), (2, 0);
-> Solusi: [0, 9, 1, 7, 8, 4, 3, 5, 2, 6, 0]; Jarak total = 43
Semua Pergerakan sudah dieksplorasi. Perbaikan tidak ditemukan. Berhenti

Solusi terbaik: [0, 9, 1, 7, 8, 4, 3, 5, 6, 2, 0]; Jarak total = 40
```

Ilustrasi Local Search untuk Masalah Penjual Keliling dengan Operator 2-opt dan Strategi Perbaikan Pertama


```
Metode: Local Search
Operator: 2-Opt
Strategi: Perbaikan Pertama (First Improvement)
Metode Penentuan Solusi Awal: Nearest Neighbor

Solusi awal: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0]; Jarak total = 70

Solusi awal: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0]; Jarak total = 70

Iterasi 1:
Solusi terbaik saat ini: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0], jarak total = 70
Pergerakan - Indeks-indeks: (0, 2); Busur-busur yang dihapus: (0, 9), (1, 7)
-> Solusi: [0, 1, 9, 7, 2, 6, 4, 3, 5, 8, 0]; Jarak total = 102
Pergerakan - Indeks-indeks: (0, 3); Busur-busur yang dihapus: (0, 9), (7, 2)
-> Solusi: [0, 7, 1, 9, 2, 6, 4, 3, 5, 8, 0]; Jarak total = 103
Pergerakan - Indeks-indeks: (0, 4); Busur-busur yang dihapus: (0, 9), (2, 6)
...
...
...
Pergerakan - Indeks-indeks: (3, 9); Busur-busur yang dihapus: (7, 2), (8, 0)
-> Solusi: [0, 9, 1, 7, 8, 5, 3, 4, 6, 2, 0]; Jarak total = 66
Perbaikan pertama ditemukan.
Solusi terbaik ditemukan: [0, 9, 1, 7, 8, 5, 3, 4, 6, 2, 0]; Jarak total = 66
Eksplorasi mulai kembali dari solusi terbaik saat ini.
```

```
Iterasi 2:
Solusi terbaik saat ini: [0, 9, 1, 7, 8, 5, 3, 4, 6, 2, 0], jarak total = 66
Pergerakan - Indeks-indeks: (0, 2); Busur-busur yang dihapus: (0, 9), (1, 7)
-> Solusi: [0, 1, 9, 7, 8, 5, 3, 4, 6, 2, 0]; Jarak total = 98
Pergerakan - Indeks-indeks: (0, 3); Busur-busur yang dihapus: (0, 9), (7, 8)
-> Solusi: [0, 7, 1, 9, 8, 5, 3, 4, 6, 2, 0]; Jarak total = 97
...
...
...
Pergerakan - Indeks-indeks: (3, 5); Busur-busur yang dihapus: (7, 8), (5, 3)
-> Solusi: [0, 9, 1, 7, 5, 8, 3, 4, 6, 2, 0]; Jarak total = 64
Perbaikan pertama ditemukan.
Solusi lebih baik ditemukan: [0, 9, 1, 7, 5, 8, 3, 4, 6, 2, 0], jarak total = 64
Eksplorasi mulai kembali dari solusi terbaik saat ini.

Iterasi 3:
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 8, 3, 4, 6, 2, 0]; jarak total = 64
Pergerakan - Indeks-indeks: (0, 2); Busur-busur yang dihapus: (0, 9), (1, 7)
-> Solusi: [0, 1, 9, 7, 5, 8, 3, 4, 6, 2, 0]; Jarak total = 96
Pergerakan - Indeks-indeks: (0, 3); Busur-busur yang dihapus: (0, 9), (7, 5)
-> Solusi: [0, 7, 1, 9, 5, 8, 3, 4, 6, 2, 0]; Jarak total = 99
...
...
...
Pergerakan - Indeks-indeks: (1, 4); Busur-busur yang dihapus: (9, 1), (5, 8)
--> Solusi: [0, 9, 5, 7, 1, 8, 3, 4, 6, 2, 0]; Jarak total = 61
Perbaikan pertama ditemukan.
Solusi lebih baik ditemukan: [0, 9, 5, 7, 1, 8, 3, 4, 6, 2, 0]; Jarak total = 61
Eksplorasi mulai kembali dari solusi terbaik saat ini.
```

```
Iterasi 4:
Solusi terbaik saat ini: [0, 9, 5, 7, 1, 8, 3, 4, 6, 2, 0]; jarak total = 61
Pergerakan - Indeks-indeks: (0, 2); Busur-busur yang dihapus: (0, 9), (5, 7)
-> Solusi: [0, 5, 9, 7, 1, 8, 3, 4, 6, 2, 0]; Jarak total = 81
Pergerakan - Indeks-indeks: (0, 3); Busur-busur yang dihapus: (0, 9), (7, 1)
-> Solusi: [0, 7, 5, 9, 1, 8, 3, 4, 6, 2, 0]; Jarak total = 74
...
...
...
Pergerakan - Indeks-indeks: (6, 9); Busur-busur yang dihapus: (3, 4), (2, 0)
-> Solusi: [0, 9, 5, 7, 1, 8, 3, 2, 6, 4, 0]; Jarak total = 51
Perbaikan pertama ditemukan.
Eksplorasi mulai kembali dari solusi terbaik saat ini.

Iterasi 5:
Solusi terbaik saat ini: [0, 9, 5, 7, 1, 8, 3, 2, 6, 4, 0], jarak total = 51
Pergerakan - Indeks-indeks: (0, 2); Busur-busur yang dihapus: (0, 9), (5, 7)
-> Solusi: [0, 5, 9, 7, 1, 8, 3, 2, 6, 4, 0]; Jarak total = 71
Pergerakan - Indeks-indeks: (0, 3); Busur-busur yang dihapus: (0, 9), (7, 1)
-> Solusi: [0, 7, 5, 9, 1, 8, 3, 2, 6, 4, 0]; Jarak total = 64
...
...
...
Pergerakan - Indeks-indeks: (7, 9); Busur-busur yang dihapus: (2, 6), (4, 0)
--> Solusi: [0, 9, 5, 7, 1, 8, 3, 2, 4, 6, 0]; Jarak total = 65
Pergerakan terakhir sudah dieksplorasi. Perbaikan tidak ditemukan. Berhenti

Solusi terbaik: [0, 9, 5, 7, 1, 8, 3, 2, 6, 4, 0]; Jarak total = 51
```

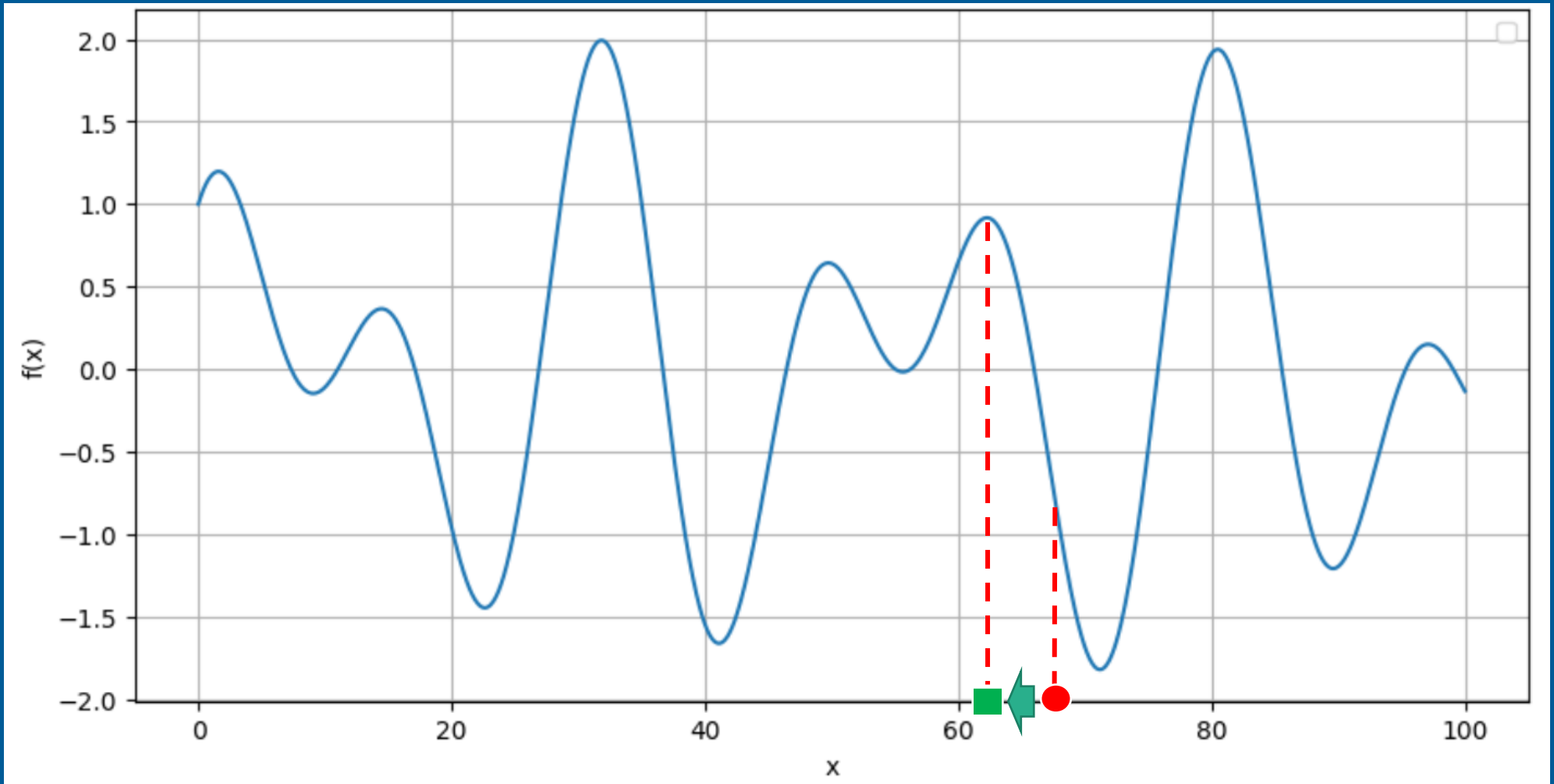
Metode Metaheuristik

Metode Metaheuristik

- Metode heuristik perbaikan seperti local search memiliki kelemahan utama, yaitu kemungkinan besar dapat terjebak pada solusi optimal lokal, bukan solusi optimal global.
- Untuk mengatasi hal tersebut, dikembangkan metode yang lebih canggih yang disebut metode metaheuristik.
- Metode metaheuristik merupakan pendekatan optimisasi yang bertujuan untuk memperluas ruang pencarian solusi dan meningkatkan peluang menemukan solusi global yang lebih baik.
- Metode ini tidak hanya memperbaiki solusi yang ada, tetapi juga melakukan eksplorasi secara adaptif untuk keluar dari jebakan solusi lokal.

Ilustrasi Jebakan Optimum Lokal (Kasus Maksimisasi)

Solusi Awal
Solusi Akhir



Klasifikasi Metode Metaheuristik

- Jumlah solusi dalam populasi
  - Solusi tunggal (single-solution based): menggunakan satu solusi yang terus diperbaiki secara iteratif
  - Solusi majemuk (population-based): menggunakan sekumpulan solusi (populasi).
- Sifat pencarian:
  - Deterministik: hasilnya cenderung sama bila dijalankan dengan kondisi awal yang sama.
  - Probabilistik: mengandung unsur acak (stochastic) untuk menjaga keberagaman solusi dan menghindari jebakan lokal.
- Pendekatan pencarian:
  - Metode konstruksi: membangun solusi secara bertahap dari awal.
  - Metode perbaikan: memulai dari solusi awal dan memperbaikinya

Beberapa Metode Metaheuristik

- Adaptive Large Neighborhood Search (ALNS)
- Ant Colony Optimization (ACO)
- Artificial Bee Colony (ABC)
- Bat Algorithm (BA)
- Butterfly Optimization Algorithm (BOA)
- Differential Evolution (DE)
- Evolutionary Strategy (ES)
- Firefly Algorithm (FA)
- Genetic Algorithm (GA)
- Greedy Randomized Adaptive Search Procedure (GRASP)
- Grey Wolf Algorithm (GWA)
- Guided Local Search (GLS)
- Harmonic Search (HS)
- Iterated Local Search (ILS)
- Multistart Local Search (MLS)
- Particle Swarm Optimization (PSO)
- Scatter Search (SS)
- Simulated Annealing (SA)
- Tabu Search (TS)
- Variable Neighborhood Search (VNS)
- Variable Neigborhood Descent (VND)

Taksonomi Beberapa Metode Metaheuristik

		Deterministik		Probabilistik	
		Konstruksi	Perbaikan	Konstruksi	Perbaikan
Solusi tunggal (Trayektori)			VND, TS, GLS	GRASP	MLS, SA ILS, VNS, ALNS
Solusi majemuk (populasi)	Berbasis Evolusi (Evolution-based)		SS		ES, GA DE, HS
	Berbasis Kawanan (Swarm-Based)			ACO	PSO, FA, ABC, BAT, BOA, GWO

Penerapan Metode Metaheuristik

Metode	Numerik	Kombinatorial
ABC	v	o
ACO	x	v
ALNS	x	v
BA	v	o
BOA	v	o
DE	v	o
ES	v	v
GA	v	v
GLS	x	v
GRASP	x	v
GWA	v	o
HS	v	o
ILS	x	v
MLS	x	v
PSO	v	o
SA	v	v
SS	v	o
TS	x	v
VND	x	v
VNS	x	v

v Sesuai
o Perlu penyesuaian
x Tidak sesuai

Ciri-Ciri Umum Metode Metaheuristik

- Secara umum, berbasis heuristik perbaikan, tetapi dilengkapi mekanisme eksplorasi global.
- Tidak menjamin solusi optimal global, tetapi mampu memberikan solusi berkualitas tinggi dalam waktu komputasi yang wajar.
- Fleksibel, dapat diterapkan pada berbagai jenis masalah optimisasi.
- Secara umum, menggabungkan elemen deterministik dan probabilistik dalam proses pencarian solusi.

Simulated Annealing

Simulated Annealing

- Metode metaheuristik yang berangkat dari analogi proses annealing pada logam dalam ilmu material.
- Pada proses annealing, logam dipanaskan hingga suhu tinggi agar atom-atomnya dapat bergerak bebas, kemudian didinginkan secara perlahan agar struktur kristal logam menjadi lebih stabil dan kuat.
- Prinsip ini diadaptasi ke dalam proses optimisasi.
  - Pada suhu tinggi, prosedur diizinkan menjelajahi berbagai solusi (termasuk yang kurang baik) untuk menghindari jebakan solusi lokal.
  - Pendinginan bertahap, prosedur secara perlahan diarahkan menuju solusi terbaik
  - Pada suhu tinggi, probabilitas penerimaan tinggi sehingga solusi yang lebih buruk masih mungkin diterima (ini membantu eksplorasi ruang solusi)
  - Ketika suhu menurun, probabilitas penerimaan semakin kecil sehingga hanya solusi yang lebih baik yang cenderung diterima (prosedur fokus pada eksploitasi untuk menemukan solusi terbaik).

Langkah-Langkah Simulated Annealing

Langkah 1: Mulai dari solusi awal x^0.

Langkah 2: Inisialisasi solusi terbaik saat ini, $x^* = x^0$.

Langkah 3: Inisialisasi calon solusi terbaik saat ini, $x' = x^0$.

Langkah 4: Inisialisasi suhu $T = T^0$.

Langkah 5: Tetapkan $iterasi = 1$

Langkah 6: Bentuk solusi (tetangga) secara random x'' berdasarkan x'.

Langkah 7: Tetapkan $\Delta = f(x'') - f(x')$ (kasus minimisasi).

Langkah 8: Jika $\Delta < 0$ (kasus minimisasi), tetapkan $x' = x''$ dan lanjutkan ke langkah 9. Sebaliknya, lanjutkan ke langkah 10.

Langkah 9: Jika $f(x') < f(x^*)$ (kasus minimisasi), tetapkan $x^* = x'$. Lanjutkan ke langkah 11.

Langkah 10: Bangkitkan bilangan acak r. Jika $r \le e^{-\Delta/T}$, tetapkan $x' = x''$.

Langkah 11: Tetapkan $iterasi \le iterasi + 1$.

Langkah 12: Jika $iterasi \le iterasimax$, kembali ke 6.

Langkah 13: Perbarui suhu $T = \alpha T$.

Langkah 14: Kembali ke langkah 5 hingga $T < T^f$. Berhenti.

Contoh Penerapan Simulated Annealing untuk Pemecahan Masalah Penjual Keliling

Tabel Jarak

Dari simpul \ Ke simpul	0	1	2	3	4	5	6	7	8	9
0	0	9	14	10	20	11	15	17	8	3
1	8	0	17	4	13	8	5	1	5	14
2	8	17	0	16	10	14	3	8	4	5
3	9	19	1	0	8	4	19	6	14	12
4	10	5	19	10	0	15	12	19	17	7
5	16	19	1	17	2	0	2	3	18	10
6	13	17	3	18	8	18	0	4	17	5
7	8	6	4	16	11	6	17	0	4	11
8	17	15	20	2	2	7	12	8	0	17
9	17	2	6	8	20	10	6	15	4	0

Fitur-Fitur dan Parameter SA:

- Operator pembentukan solusi baru:
  - Operator pertukaran
- Nilai-nilai parameter:
  - Suhu awal T^0 = 10
  - Suhu akhir T^f = 1 (kriteria henti)
  - *iterasimax* = 10 (jumlah iterasi maksimum pada tiap level suhu)
  - Tingkat pendinginan α = 0,9.

```
Suhu: 10.0000

Iterasi 1
Solusi terbaik saat ini: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0], jarak total = 70
Calon solusi terbaik saat ini: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0], jarak total = 70
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (1, 6); simpul-simpul yang dipertukarkan: (9, 4); solusi: [0, 4, 1, 7, 2, 6, 9, 3, 5, 8, 0], jarak total = 85
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.7305 > probabilitas penerimaan (0.2231) -> solusi baru tidak diterima sebagai calon solusi terbaik yang baru

Iterasi 2
Solusi terbaik saat ini: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0], jarak total = 70
Calon solusi terbaik saat ini: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0], jarak total = 70
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (1, 9); simpul-simpul yang dipertukarkan: (9, 8); solusi: [0, 8, 1, 7, 2, 6, 4, 3, 5, 9, 0], jarak total = 80
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.7601 > probabilitas penerimaan (0.3679) -> solusi baru tidak diterima sebagai calon solusi terbaik yang baru

Iterasi 3
Solusi terbaik saat ini: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0], jarak total = 70
Calon solusi terbaik saat ini: [0, 9, 1, 7, 2, 6, 4, 3, 5, 8, 0], jarak total = 70
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (4, 8); simpul-simpul yang dipertukarkan: (2, 5); solusi: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Solusi baru lebih baik dari calon solusi terbaik saat ini
Calon solusi terbaik yang baru: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik lebih baik dari solusi terbaik saat ini
Solusi terbaik yang baru: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54

Iterasi 4
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (5, 9); simpul-simpul yang dipertukarkan: (6, 8); solusi: [0, 9, 1, 7, 5, 8, 4, 3, 2, 6, 0], jarak total = 59
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.2225 <= probabilitas penerimaan (0.6065) -> solusi baru diterima sebagai calon solusi terbaik yang baru
Calon solusi terbaik yang baru: [0, 9, 1, 7, 5, 8, 4, 3, 2, 6, 0], jarak total = 59
```

```
Iterasi 5
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 9, 1, 7, 5, 8, 4, 3, 2, 6, 0], jarak total = 59
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (7, 9); simpul-simpul yang dipertukarkan: (3, 6); solusi: [0, 9, 1, 7, 5, 8, 4, 6, 2, 3, 0], jarak total = 72
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.3975 > probabilitas penerimaan (0.2725) -> solusi baru tidak diterima sebagai calon solusi terbaik yang baru

Iterasi 6
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 9, 1, 7, 5, 8, 4, 3, 2, 6, 0], jarak total = 59
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (1, 5); simpul-simpul yang dipertukarkan: (9, 8); solusi: [0, 8, 1, 7, 5, 9, 4, 3, 2, 6, 0], jarak total = 87
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.4720 > probabilitas penerimaan (0.0608) -> solusi baru tidak diterima sebagai calon solusi terbaik yang baru

Iterasi 7
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 9, 1, 7, 5, 8, 4, 3, 2, 6, 0], jarak total = 59
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (2, 4); simpul-simpul yang dipertukarkan: (1, 5); solusi: [0, 9, 5, 7, 1, 8, 4, 3, 2, 6, 0], jarak total = 56
Solusi baru lebih baik dari calon solusi terbaik saat ini
Calon solusi terbaik yang baru: [0, 9, 5, 7, 1, 8, 4, 3, 2, 6, 0], jarak total = 56

Iterasi 8
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 9, 5, 7, 1, 8, 4, 3, 2, 6, 0], jarak total = 56
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (6, 3); simpul-simpul yang dipertukarkan: (4, 7); solusi: [0, 9, 5, 4, 1, 8, 7, 3, 2, 6, 0], jarak total = 67
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.7576 > probabilitas penerimaan (0.3329) -> solusi baru tidak diterima sebagai calon solusi terbaik yang baru
```

```
Iterasi 9
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 9, 5, 7, 1, 8, 4, 3, 2, 6, 0], jarak total = 56
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (9, 6); simpul-simpul yang dipertukarkan: (6, 4); solusi: [0, 9, 5, 7, 1, 8, 6, 3, 2, 4, 0], jarak total = 78
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.0448 <= probabilitas penerimaan (0.1108) -> solusi baru diterima sebagai calon solusi terbaik yang baru
Calon solusi terbaik yang baru: [0, 9, 5, 7, 1, 8, 6, 3, 2, 4, 0], jarak total = 78

Iterasi 10
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 9, 5, 7, 1, 8, 6, 3, 2, 4, 0], jarak total = 78
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (4, 9); simpul-simpul yang dipertukarkan: (1, 4); solusi: [0, 9, 5, 7, 4, 8, 6, 3, 2, 1, 0], jarak total = 100
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.1043 <= probabilitas penerimaan (0.1108) -> solusi baru diterima sebagai calon solusi terbaik yang baru
```

```
Pembaruan suhu:
Suhu: 9.0000

Iterasi 1
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 9, 5, 7, 4, 8, 6, 3, 2, 1, 0], jarak total = 100
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (2, 7); simpul-simpul yang dipertukarkan: (5, 3); solusi: [0, 9, 3, 7, 4, 8, 6, 5, 2, 1, 0], jarak total = 101
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.7040 <= probabilitas penerimaan (0.8948) -> solusi baru diterima sebagai calon solusi terbaik yang baru
Calon solusi terbaik yang baru: [0, 9, 3, 7, 4, 8, 6, 5, 2, 1, 0], jarak total = 101

...

Iterasi 10
Solusi terbaik saat ini: [0, 9, 1, 7, 5, 6, 4, 3, 2, 8, 0], jarak total = 54
Calon solusi terbaik saat ini: [0, 6, 7, 1, 4, 3, 5, 8, 2, 9, 0], jarak total = 112
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (1, 6); simpul-simpul yang dipertukarkan: (6, 5); solusi: [0, 5, 7, 1, 4, 3, 6, 8, 2, 9, 0], jarak total = 121
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.8385 > probabilitas penerimaan (0.3679) -> solusi baru tidak diterima sebagai calon solusi terbaik yang baru
```

```
Pembaruan suhu
Suhu: 1.0942

Iterasi 1
Solusi terbaik saat ini: [0, 9, 8, 4, 1, 3, 5, 2, 6, 7, 0], jarak total = 38
Calon solusi terbaik saat ini: [0, 9, 8, 4, 1, 3, 5, 6, 7, 2, 0], jarak total = 41
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (3, 8); simpul-simpul yang dipertukarkan: (4, 7); solusi: [0, 9, 8, 7, 1, 3, 5, 6, 4, 2, 0], jarak total = 68
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.4128 > probabilitas penerimaan (0.0000) -> solusi baru tidak diterima sebagai calon solusi terbaik yang baru

...

Iterasi 10
Solusi terbaik saat ini: [0, 9, 8, 4, 1, 3, 5, 2, 6, 7, 0], jarak total = 38
Calon solusi terbaik saat ini: [0, 9, 8, 4, 1, 3, 5, 6, 7, 2, 0], jarak total = 41
Pembentukan solusi random (operator pertukaran) berdasarkan calon solusi terbaik saat ini
--> Pasangan indeks: (5, 1); simpul-simpul yang dipertukarkan: (3, 9); solusi: [0, 3, 8, 4, 1, 9, 5, 6, 7, 2, 0], jarak total = 74
Solusi baru tidak lebih baik dari solusi calon terbaik saat ini
Bilangan random 0.2542 > probabilitas penerimaan (0.0000) -> solusi baru tidak diterima sebagai calon solusi terbaik yang baru



Pembaruan suhu
Suhu: 0.9848 < suhu akhir (1) --> berhenti

Solusi terbaik:
Solusi: [0, 9, 8, 4, 1, 3, 5, 2, 6, 7, 0]
Jarak total = 38
```

Genetic Algorithm

Genetic Algorithm

- Metode metaheuristik yang berbasis populasi yang terinspirasi dari mekanisme evolusi biologis melalui proses seleksi alam.
- Individu-individu dengan kemampuan bertahan (fitness) lebih baik akan memiliki peluang lebih besar untuk berkembang biak dan menurunkan sifatnya ke generasi berikutnya.
- Dalam konteks optimisasi, setiap individu dalam populasi berasosiasi dengan solusi.
- Solusi-solusi dengan kemampuan bertahan (berasosiasi dengan nilai fungsi tujuan) lebih baik akan memiliki peluang lebih besar untuk berkembang biak dan menurunkan sifatnya ke generasi berikutnya.
- Metode metaheuristik berbasis probabilistik, karena proses eksplorasi ruang solusi dilakukan secara acak namun terarah melalui penggunaan operator seleksi, persilangan (crossover), dan mutasi (mutation).

Langkah-Langkah

1. Bangkitkan populasi awal.
2. Lakukan prosedur dekoding (konversi tiap individu ke solusi asli dari masalah) dan hitung nilai fungsi tujuan yang berasosiasi dengan tiap solusi.
3. Hitung nilai fitnes yang berasosiasi dengan tiap solusi.
4. Tetapkan solusi terbaik hingga generasi (iterasi) saat ini.
5. Bentuk populasi baru dengan mekanisme seleksi, persilangan, dan mutasi.
6. Ulangi langkah 2-5 hingga generasi mencapai jumlah generasi maksimum yang ditetapkan.

Representasi Individu

- Bekerja menggunakan sejumlah individu (kromosom).
- Setiap individu merepresentasikan satu kandidat solusi dari permasalahan optimisasi yang sedang diselesaikan.
- Individu direpresentasikan dalam suatu format kode tertentu yang disesuaikan dengan karakteristik masalah.
- Setiap elemen pada individu disebut gen (gene), dan posisi gen dalam individu menentukan bagian dari solusi yang diwakilinya.
- Untuk kasus representasi biner, nilai gen disebut bit, dengan kemungkinan nilai 0 atau 1.
- Panjang individu (jumlah gen) bergantung pada dimensi dan kompleksitas masalah optimisasi yang dihadapi, serta jenis representasi yang digunakan.
- Pemilihan bentuk representasi yang tepat sangat penting karena akan memengaruhi cara penerapan operator genetika seperti persilangan (crossover) dan mutasi (mutation).

Jenis Representasi Individu

- Kode Biner
- Permutasi
- Bilangan riil

Representasi Kode Biner

- Representasi yang paling klasik
- Representasi dalam bentuk deretan angka 0 atau 1
- Cocok untuk masalah optimisasi yang melibatkan variabel diskret, logika keputusan, atau masalah yang mudah dikodekan ke dalam bentuk biner.
- Bilangan riil pun dapat direpresentasikan dengan string biner melalui proses binary-to-decimal decoding, dengan sejumlah bit tertentu merepresentasikan satu nilai bilangan riil.
- Dalam representasi string biner, istilah gen disebut bit
- Jumlah bit dalam individu disesuaikan dengan masalah,

Berikut ini adalah contoh representasi individu dalam string biner untuk satu variabel yang nilainya merupakan bilangan bulat antara 0 dan 200. Nilai batas atas = 200, berada antara $2^7 = 128$ dan $2^8 = 256$. Dengan panjang individu (jumlah bit) yang diperlukan adalah 8. Salah individu adalah berikut ini.

```
[0, 1, 1, 0, 0, 1, 1, 1]
```

Berikut ini adalah contoh representasi individu dalam string biner untuk satu variabel yang nilainya merupakan bilangan riil antara 0 dan 200. Misal tingkat ketelitian yang dikehendaki adalah 6 angka di belakang koma (atau $1/1000000$). Jumlah elemen adalah $(200-0)\times 1000000 = 200000000$. Nilai ini berada berada antara $2^{27} = 134217728$ dan $2^{28} = 268435456$. Dengan demikian, panjang individu (jumlah bit) yang diperlukan adalah 28. Salah individu adalah berikut ini

```
[0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1]
```

Representasi dalam Bentuk Permutasi

- Representasi untuk solusi masalah kombinatorial seperti masalah penjual keliling, masalah rute kendaraan, atau masalah penjadwalan
- Permutasi (deretan) dari angka/huruf dengan tiap angka/huruf muncul sekali

Berikut ini adalah contoh representasi individu untuk masalah penjual keliling yang memiliki 9 titik (selain titik depot) dalam bentuk permutasi. Panjang individu adalah 9 (sembilan).

```
[3, 2, 7, 1, 4, 8, 6, 5, 9]
```

Representasi dalam Bentuk Bilangan Riil

- Pada masalah optimisasi yang melibatkan variabel kontinu, representasi biner sering kali kurang efisien karena memerlukan proses konversi dan presisi terbatas.
- Representasi langsung dalam bentuk bilangan riil (real numbers), di mana setiap gen menunjukkan nilai numerik sebenarnya dari variabel keputusan.

Contoh representasi individu untuk satu variabel yang bernilai kontinu adalah:

```
97,0717
```

Contoh representasi individu untuk dua variabel yang bernilai kontinu adalah:

```
[17,1268,65,0617]
```

Proses Dekoding

- Menerjemahkan representasi individu ke dalam bentuk solusi asli dari masalah

Contoh Dekoding Representasi Biner ke Bilangan Bulat

Misal diberikan individu dengan panjang individu n dan tiap gen j bernilainya b_j.

Konversi ke variabel x yang bernilai bulat dinyatakan dengan:

$$x = \sum_{j=0}^{n-1} 2^{n-j-1} b_j$$

Hasil Dekoding

```
Individu 1:  [1, 0, 1, 0, 0, 1, 0, 0], x = 164
Individu 2:  [0, 1, 1, 0, 0, 1, 1, 0], x = 102
Individu 3:  [1, 1, 0, 0, 1, 1, 1, 1], x = 207
Individu 4:  [0, 1, 0, 1, 1, 1, 0, 1], x = 93
Individu 5:  [1, 1, 0, 0, 0, 1, 0, 1], x = 197
Individu 6:  [1, 1, 1, 0, 0, 1, 0, 1], x = 229
Individu 7:  [0, 0, 1, 0, 1, 1, 0, 1], x = 45
Individu 8:  [1, 1, 1, 1, 1, 0, 0, 0], x = 248
Individu 9:  [0, 1, 1, 1, 0, 0, 1, 1], x = 115
Individu 10: [1, 0, 1, 0, 0, 1, 1, 0], x = 166
```

Contoh Dekoding Representasi Biner ke Bilangan Riil

Misal diberikan individu dengan panjang individu n dan tiap gen j bernilainya b_j. Konversi ke variabel x yang bernilai riil (kontinu) dengan batas-batas bawah dan atas masing-masing adalah $x_{\min}$ dan $x_{\max}$ dinyatakan dengan:

$$x = x_{\min} + x' \left(\frac{x_{\max} - x_{\min}}{2^n - 1} \right)$$

dengan

$$x' = \sum_{j=0}^{n-1} 2^{n-j-1} b_j$$

```
Hasil Dekoding
String biner -> Nilai riil (kontinu)
Jumlah bit = 28
Batas bawah x = 0
Batas atas x = 200

Individu 1:  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1] -> x = 199.8182
Individu 2:  [0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1] -> x = 41.6213
Individu 3:  [1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0] -> x = 129.6948
Individu 4:  [0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1] -> x = 27.8859
Individu 5:  [1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1] -> x = 125.2280
Individu 6:  [0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1] -> x = 8.3581
Individu 7:  [0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0] -> x = 88.7056
Individu 8:  [1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0] -> x = 142.0831
Individu 9:  [1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1] -> x = 116.8825
Individu 10: [0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0] -> x = 22.8423
```

Contoh Dekoding untuk Representasi Permutasi ke Solusi Masalah Penjual Keliling

- Proses dekoding untuk mengkonversi individu ke bentuk asli dari masalah penjual keliling dilakukan dengan penambahan angka 0 (yang menunjukkan depot) pada awal dan akhir permutasi.

Hasil Dekoding untuk Masalah Penjual Keliling

```
Individu 1:     Solusi: [0, 5, 4, 3, 6, 1, 9, 7, 8, 2, 0]
Individu 2:     Solusi: [0, 1, 4, 6, 3, 2, 7, 9, 8, 5, 0]
Individu 3:     Solusi: [0, 1, 5, 2, 7, 6, 4, 8, 3, 9, 0]
Individu 4:     Solusi: [0, 4, 3, 5, 1, 9, 8, 6, 7, 2, 0]
Individu 5:     Solusi: [0, 8, 4, 1, 6, 2, 5, 3, 9, 7, 0]
Individu 6:     Solusi: [0, 9, 6, 8, 1, 7, 5, 2, 3, 4, 0]
Individu 7:     Solusi: [0, 5, 6, 2, 1, 7, 3, 4, 9, 8, 0]
Individu 8:     Solusi: [0, 6, 9, 4, 5, 7, 8, 1, 2, 3, 0]
Individu 9:     Solusi: [0, 1, 7, 4, 2, 9, 5, 3, 8, 6, 0]
Individu 10:    Solusi: [0, 3, 7, 1, 4, 8, 2, 5, 6, 9, 0]
```

Penentuan Nilai Fitness

- Nilai fungsi tujuan (objective function) dihitung untuk setiap solusi.
- Nilai ini mencerminkan kualitas atau kinerja solusi terhadap permasalahan yang sedang diselesaikan.
- Berdasarkan nilai fungsi tujuan tersebut, dilakukan perhitungan nilai fitnes (fitness value) untuk setiap individu.
- Secara umum, nilai fitness identik dengan nilai fungsi tujuan, terutama pada masalah maksimisasi.
- Namun, pada beberapa kasus, nilai fitness dapat dikonversi agar semua nilai fitness taknegatif atau dinormalisasi, misalnya untuk masalah minimisasi atau ketika diperlukan untuk mekanisme seleksi yang proporsional (misal menggunakan metode seleksi roda rolet).

Fungsi fitness (secara umum)

$$g(x_i) = f(x_i)$$

Fungsi fitness (kasus minimisasi)

$$g(x_i) = \frac{1}{f(x_i)}$$

Normalisasi (agar nilai fitness > 0)

$$g'(x_i) = g(x_i) + c$$

Hasil Dekoding, Nilai Fungsi Tujuan, dan Nilai Fitnes
Untuk solusi yang taklayak, x < 0 atau x > 200, tambahkan penalti (-100) pada nilai fungsi tujuan
Nilai fitness = Nilai fungsi tujuan + 1000 (agar nilai fitness > 0)

Individu 1: [1, 0, 1, 0, 0, 1, 0, 0], x = 164, Nilai fungsi tujuan = -0.1586, Nilai fitnes = 999.8414
Individu 2: [0, 1, 1, 0, 0, 1, 1, 0], x = 102, Nilai fungsi tujuan = -0.3480, Nilai fitnes = 999.6520
Individu 3: [1, 1, 0, 0, 1, 1, 1, 1], x = 207, Nilai fungsi tujuan = -98.0798, Nilai fitnes = 901.9202
Individu 4: [0, 1, 0, 1, 1, 1, 0, 1], x = 93, Nilai fungsi tujuan = -0.5691, Nilai fitnes = 999.4309
Individu 5: [1, 1, 0, 0, 0, 1, 0, 1], x = 197, Nilai fungsi tujuan = -1.2324, Nilai fitnes = 998.7676
Individu 6: [1, 1, 1, 0, 0, 1, 0, 1], x = 229, Nilai fungsi tujuan = -99.7374, Nilai fitnes = 900.2626
Individu 7: [0, 0, 1, 0, 1, 1, 0, 1], x = 45, Nilai fungsi tujuan = -0.5851, Nilai fitnes = 999.4149
Individu 8: [1, 1, 1, 1, 1, 0, 0, 0], x = 248, Nilai fungsi tujuan = -101.7392, Nilai fitnes = 898.2608
Individu 9: [0, 1, 1, 1, 0, 0, 1, 1], x = 115, Nilai fungsi tujuan = -0.0752, Nilai fitnes = 999.9248
Individu 10: [1, 0, 1, 0, 0, 1, 1, 0], x = 166, Nilai fungsi tujuan = -1.3197, Nilai fitnes = 998.6803

Solusi, Nilai fungsi tujuan (jarak total), dan Nilai fitness untuk Masalah Penjual Keliling
Nilai fitness = 1/Jarak total

```
Individu 1:       Solusi: [0, 5, 4, 3, 6, 1, 9, 7, 8, 2, 0], Jarak total = 121, Nilai fitnes = 0.0083
Individu 2:       Solusi: [0, 1, 4, 6, 3, 2, 7, 9, 8, 5, 0], Jarak total = 99,  Nilai fitnes = 0.0101
Individu 3:       Solusi: [0, 1, 5, 2, 7, 6, 4, 8, 3, 9, 0], Jarak total = 99,  Nilai fitnes = 0.0101
Individu 4:       Solusi: [0, 4, 3, 5, 1, 9, 8, 6, 7, 2, 0], Jarak total = 100, Nilai fitnes = 0.0100
Individu 5:       Solusi: [0, 8, 4, 1, 6, 2, 5, 3, 9, 7, 0], Jarak total = 89,  Nilai fitnes = 0.0112
Individu 6:       Solusi: [0, 9, 6, 8, 1, 7, 5, 2, 3, 4, 0], Jarak total = 83,  Nilai fitnes = 0.0120
Individu 7:       Solusi: [0, 5, 6, 2, 1, 7, 3, 4, 9, 8, 0], Jarak total = 86,  Nilai fitnes = 0.0116
Individu 8:       Solusi: [0, 6, 9, 4, 5, 7, 8, 1, 2, 3, 0], Jarak total = 119, Nilai fitnes = 0.0084
Individu 9:       Solusi: [0, 1, 7, 4, 2, 9, 5, 3, 8, 6, 0], Jarak total = 111, Nilai fitnes = 0.0090
Individu 10:      Solusi: [0, 3, 7, 1, 4, 8, 2, 5, 6, 9, 0], Jarak total = 110, Nilai fitnes = 0.0091
```

Seleksi Roda Rolet

- Memberikan probabilitas seleksi yang proporsional terhadap nilai fitness setiap individu.
- Prosesnya dianalogikan seperti roda rolet, di mana setiap individu mendapatkan porsi area roda yang sebanding dengan nilai fitnes-nya.
- Semakin tinggi nilai fitnes suatu individu, semakin besar kemungkinan terpilih menjadi induk.

$$p(x_i) = \frac{g(x_i)}{\sum_{i=1}^{n} g(x_i)}$$

Seleksi Turnamen

- Sejumlah individu dipilih secara acak dari populasi untuk membentuk kelompok turnamen (misalnya berukuran dua atau tiga individu).
- Nilai fitness dari individu-individu tersebut dibandingkan, dan individu dengan nilai fitnes terbaik dalam kelompok turnamen dinyatakan sebagai pemenang yang akan terpilih sebagai induk.
- Kelebihan:
  - Lebih sederhana dan tidak memerlukan normalisasi nilai fitness
  - Memungkinkan pengendalian tekanan seleksi melalui ukuran turnamen (semakin besar ukuran turnamen, semakin kuat tekanan seleksinya)

```
Probabilitas Seleksi:

Penggunaan untuk metode seleksi dengan roda rolet
NF      : Nilai Fitnes (dengan penyesuaian agar NF taknegatif)
Prob    : Probabilitas
ProbKum: Probabilitas Kumulatif

Individu 1:   NF = 1000.6211   Prob. = 0.1111   Prob. Kumulatif = 0.1111
Individu 2:   NF = 1001.6070   Prob. = 0.1112   Prob. Kumulatif = 0.2223
Individu 3:   NF = 1001.3041   Prob. = 0.1112   Prob. Kumulatif = 0.3334
Individu 4:   NF = 1001.5825   Prob. = 0.1112   Prob. Kumulatif = 0.4446
Individu 5:   NF = 1001.2566   Prob. = 0.1111   Prob. Kumulatif = 0.5557
Individu 6:   NF = 0.0001      Prob. = 0.0000   Prob. Kumulatif = 0.5557
Individu 7:   NF = 1000.5380   Prob. = 0.1111   Prob. Kumulatif = 0.6668
Individu 8:   NF = 1001.0820   Prob. = 0.1111   Prob. Kumulatif = 0.7779
Individu 9:   NF = 999.4660    Prob. = 0.1109   Prob. Kumulatif = 0.8889
Individu 10:  NF = 1000.9568   Prob. = 0.1111   Prob. Kumulatif = 1.0000


        Seleksi Roda Rolet

        Bilangan acak: 0.4524 (antara 0.4446 dan 0.5557)
        Terpilih -> Individu 5:  [0, 0, 0, 0, 0, 1, 1, 0]
        Induk 1: [0, 0, 0, 0, 0, 1, 1, 0]

        Bilangan acak: 0.5598 (antara 0.5557 dan 0.6668)
        Terpilih -> Individu 7:  [1, 0, 0, 0, 1, 1, 0, 0]
        Induk 2: [1, 0, 0, 0, 1, 1, 0, 0]
```

```
Seleksi Turnamen
Ukuran Turnamen = 3
Nilai Fitness = Nilai Fungsi Tujuan (tanpa penyesuaian)

Calon individu yang dipilih secara acak dari populasi yang dimasukkan ke turnamen:
Individu 8 : [0, 1, 1, 0, 0, 0, 1, 0] Nilai Fitnes = 0.1157
Individu 10: [0, 0, 1, 1, 1, 0, 0, 0] Nilai Fitnes = -0.0094
Individu 4 : [0, 1, 1, 0, 1, 0, 1, 1] Nilai Fitnes = 0.6162
Individu terbaik (kasus maksimisasi) -> Individu 4
Induk 1    : [0, 1, 1, 0, 1, 0, 1, 1]

Calon individu yang dipilih secara acak dari populasi yang dimasukkan ke turnamen:
Individu 3: [1, 0, 1, 1, 1, 1, 1, 1] Nilai Fitnes = 0.3379
Individu 9: [0, 1, 1, 1, 1, 0, 1, 0] Nilai Fitnes = -1.5002
Individu 8: [0, 1, 1, 0, 0, 0, 1, 0] Nilai Fitnes = 0.1157
Individu terbaik (kasus maksimisasi) -> Individu 3
Induk 2   : [1, 0, 1, 1, 1, 1, 1, 1]
```

Persilangan

- Proses perkawinan dari dua induk untuk mewariskan gen-gen ke keturunannya (anak-anaknya)
- Dari kedua induk yang memiliki fitnes yang baik, anak-anak yang dihasilkan diharapkan memiliki fitness yang baik juga.
- Terjadi atau tidaknya persilangan (perkawinan) dikendalikan oleh parameter tingkat persilangan.
- Mengikuti analogi dari seleksi alam, tingkat persilangan umumnya memiliki nilai yang cukup besar (misal, 0,7-0,8)
- Jika persilangan terjadi, maka kedua induk mewariskan sebagai gen-gennya kepada tiap kedua anaknya.
- Jika persilangan tidak terjadi, maka individu dari tiap induk disalin langsung ke anak,

Beberapa Operator Persilangan untuk Representasi Kode Biner

- Persilangan Satu-Titik (One-Point Crossover)
- Persilangan Dua-Titik (Two-Point Crossover)
- Persilangan Seragam (Uniform Crossover)

Persilangan Satu-Titik

- Mengombinasikan gen dari dua induk dengan tetap mempertahankan urutan atau posisi segmen gen tertentu.
- Persilangan yang paling sederhana dan banyak digunakan dalam algoritma genetika.
- Prosedur pembentukan anak pertama dilakukan melalui langkah-langkah berikut:
  - Tentukan satu titik potong secara acak untuk membagi individu kedua induk menjadi dua segmen, yaitu segmen kepala (bagian kiri) dan segmen ekor (bagian kanan).
  - Bentuk anak pertama dengan menyalin segmen kepala dari induk pertama, kemudian menyalin segmen ekor dari induk kedua.
- Pembentukan anak kedua dilakukan dengan cara yang sama, tetapi dengan menukar peran induk pertama dan induk kedua.

```
Persilangan Satu-Titik (One-Point Crossover)
Catatan: indeks dimulai dari 0

Induk-induk hasil seleksi:
  Induk 1: [1, 1, 0, 0, 1, 1, 0, 1]
  Induk 2: [1, 1, 0, 1, 0, 1, 1, 0]

Penentuan titik potong secara acak
  Titik potong: 4

Pembentukan anak 1:
Inisialisasi
  Anak 1:  [None, None, None, None, None, None, None, None]
Penyalinan segmen kepala dari induk 1, [1, 1, 0, 0]
  Anak 1:  [1, 1, 0, 0, None, None, None, None]
Penyalinan segmen ekor dari induk 2, [0, 1, 1, 0]
  Anak 1:  [1, 1, 0, 0, 0, 1, 1, 0]

Pembentukan anak 2:
Inisialisasi
  Anak 2:  [None, None, None, None, None, None, None, None]
Penyalinan segmen kepala dari induk 2, [1, 1, 0, 1]
  Anak 2:  [1, 1, 0, 1, None, None, None, None]
Penyalinan segmen ekor dari induk 1, [1, 1, 0, 1]
  Anak 2:  [1, 1, 0, 1, 1, 1, 0, 1]

Anak-anak hasil persilangan:
  Anak 1:  [1, 1, 0, 0, 0, 1, 1, 0]
  Anak 2:  [1, 1, 0, 1, 1, 1, 0, 1]
```

Persilangan Dua-Titik

- Pengembangan dari operator persilangan satu-titik, di mana individu dibagi menjadi tiga segmen menggunakan dua titik potong acak.
- Memungkinkan kombinasi gen yang lebih beragam dibandingkan persilangan satu-titik, karena dua titik potong menciptakan variasi lebih banyak dalam pembentukan keturunan.
- Prosedur pembentukan anak pertama dilakukan melalui langkah-langkah berikut:
  - Tentukan dua titik potong secara acak untuk membagi individu kedua induk menjadi tiga segmen, yaitu segmen kepala (bagian kiri), segmen tengah, dan segmen ekor (bagian kanan).
  - Bentuk anak pertama dengan menyalin segmen kepala dan segmen ekor dari induk pertama, kemudian menyalin segmen tengah dari induk kedua.
- Pembentukan anak kedua dilakukan dengan cara yang sama, tetapi dengan menukar peran induk pertama dan induk kedua.

```
Persilangan Dua-Titik (Two-Point Crossover)
Catatan: indeks dimulai dari 0

Induk-induk hasil seleksi:
  Induk 1: [0, 1, 1, 1, 1, 1, 1, 0]
  Induk 2: [0, 1, 0, 1, 1, 1, 0, 0]

Tentukan dua titik potong secara acak (indeks 1 s/d n-1, n = panjang individu)
Titik potong 1 antara indeks 1 s/d n-1
Titik potong 2 antara indeks 1 s/d n-1 dengan titik potong 1 < titik potong 2)
- Titik potong 1: 2
- Titik potong 2: 4

Pembentukan anak 1:
- Salin segmen induk 1 sebelum titik potong 1 (s/d indeks 1): [0, 1]
- Salin segmen induk 1 mulai titik potong 2 (mulai indeks 4): [1, 1, 1, 0]
  Anak 1:  [0, 1, None, None, 1, 1, 1, 0]
- Salin segmen induk 2 mulai titik potong 1 (indeks 2) s/d sebelum titik potong 2 (s/d indeks 3): [0, 1]
  Anak 1:  [0, 1, 0, 1, 1, 1, 1, 0]

Pembentukan anak 2:
- Salin segmen induk 2 sebelum titik potong 1 (s/d indeks 1): [0, 1]
- Salin segmen induk 2 mulai titik potong 2 (mulai indeks 4): [1, 1, 0, 0]
  Anak 2:  [0, 1, None, None, 1, 1, 0, 0]
- Salin segmen induk 1 mulai titik potong 1 (indeks 2) s/d sebelum titik potong 2 (s/d indeks 3): [1, 1]
  Anak 2:  [0, 1, 1, 1, 1, 1, 0, 0]

Anak-anak hasil persilangan:
  Anak 1:  [0, 1, 0, 1, 1, 1, 1, 0]
  Anak 2:  [0, 1, 1, 1, 1, 1, 0, 0]
```

Persilangan Seragam

- Mengkombinasikan gen dari dua induk secara merata dan acak, tanpa mempertahankan urutan atau posisi blok gen tertentu.
- Menggunakan pola penyilangan acak yang menentukan dari induk mana setiap gen akan diturunkan kepada anak.
- Prosedur pembentukan anak pertama dalam persilangan dilakukan melalui langkah-langkah berikut:
  - Tentukan pola atau masker biner acak dengan panjang yang sama seperti individu. Setiap posisi gen memiliki probabilitas tertentu (biasanya 0,5) untuk diwariskan dari induk pertama atau induk kedua. Nilai 1 pada masker menunjukkan gen diambil dari induk pertama, sedangkan nilai 0 berarti gen diambil dari induk kedua.
  - Bentuk anak pertama dengan mengikuti pola masker tersebut. Untuk setiap posisi gen, jika nilai masker 1, maka gen diambil dari induk pertama. Jika 0, gen diambil dari induk kedua.
- Pembentukan anak kedua dapat dilakukan dengan menggunakan masker yang sama, tetapi dengan induk ditukar perannya. Ilustrasi mekanisme uniform crossover ditunjukkan berikut ini.

```
Persilangan Seragam (Uniform Crossover)
Catatan: indeks dimulai dari 0

Induk-induk hasil seleksi:
  Induk 1: [0, 1, 0, 0, 0, 0, 1, 0]
  Induk 2: [1, 0, 0, 0, 1, 1, 0, 0]

Tentukan masker secara acak:
  Masker:  [1, 1, 0, 1, 0, 1, 1, 0]

Pembentukan anak 1:
Inisialisasi
  Anak 1:  [None, None, None, None, None, None, None, None]
Penyalinan gen dari induk 1 (jika masker = 1)
  Anak 1:  [0, 1, None, 0, None, 0, 1, None]
Penyalinan gen dari induk 2 (jika masker = 0)
  Anak 1:  [0, 1, 0, 0, 1, 0, 1, 0]

Pembentukan anak 2:
Inisialisasi
  Anak 2:  [None, None, None, None, None, None, None, None]
Penyalinan gen dari induk 2 (jika masker = 1)
  Anak 2:  [1, 0, None, 0, None, 1, 0, None]
Penyalinan gen dari induk 1 (jika masker = 0)
  Anak 2:  [1, 0, 0, 0, 0, 1, 0, 0]

Anak-anak hasil persilangan:
  Anak 1:  [0, 1, 0, 0, 1, 0, 1, 0]
  Anak 2:  [1, 0, 0, 0, 0, 1, 0, 0]
```

Beberapa Operator Persilangan untuk Representasi Permutasi

- Persilangan Berbasis Posisi (Position-Based Crossover)
- Persilangan Pemetaan Parsial (Partially Mapped Crossover)

Persilangan Berbasis Posisi

- Mempertahankan posisi gen tertentu dari induk agar informasi posisi relatif tetap terjaga pada keturunannya.
- Pada pembentukan anak pertama, operator ini bekerja dengan langkah-langkah berikut:
  - Tentukan secara acak sejumlah posisi gen dari induk pertama yang akan disalin langsung ke anak pertama pada posisi yang sama.
  - Posisi-posisi yang masih kosong pada anak pertama kemudian diisi dengan gen-gen dari induk kedua yang belum ada pada anak pertama sesuai urutan kemunculannya, dimulai dari posisi paling kiri individu.
- Pembentukan anak kedua dilakukan dengan cara yang sama, tetapi dengan menukar peran induk pertama dan induk kedua.

```
Persilangan Berbasis Posisi (Position-Based Crossover)
Catatan: indeks dimulai dari 0

Induk-induk hasil seleksi:
  Induk 1: [9, 3, 6, 7, 4, 2, 1, 8, 5]
  Induk 2: [4, 8, 5, 7, 9, 3, 2, 6, 1]

Jumlah gen yang dipilih secara acak: 6
Posisi gen yang akan disalin dari induk ke anak secara acak
Indeks posisi gen: [0, 2, 3, 4, 5, 6]

Pembentukan anak 1:
Inisialisasi
  Anak 1:  [None, None, None, None, None, None, None, None, None]
Penyalinan gen sesuai posisi dari induk 1, [9, 6, 7, 4, 2, 1]
  Anak 1:  [9, None, 6, 7, 4, 2, 1, None, None]
Penyalinan gen dari induk 2
- Urutan gen pada induk 2 yang belum ada pada anak 1, [8, 5, 3]
- Isi posisi kosong pada anak 1 sesuai urutan kemunculannya
  Anak 1:  [9, 8, 6, 7, 4, 2, 1, 5, 3]

Pembentukan anak 2:
Inisialisasi
  Anak 2:  [None, None, None, None, None, None, None, None, None]
Penyalinan gen sesuai posisi dari induk 2, [4, 5, 7, 9, 3, 2]
  Anak 2:  [4, None, 5, 7, 9, 3, 2, None, None]
Penyalinan gen dari induk 1
- Urutan gen pada induk 1 yang belum ada pada anak 2, [6, 1, 8]
- Isi posisi kosong pada anak 2 sesuai urutan kemunculannya
  Anak 2:  [4, 6, 5, 7, 9, 3, 2, 1, 8]

Hasil persilangan:
  Anak 1:  [9, 8, 6, 7, 4, 2, 1, 5, 3]
  Anak 2:  [4, 6, 5, 7, 9, 3, 2, 1, 8]
```

Persilangan Pemetaan Parsial

- Mempertahankan posisi relatif serta pasangan gen antara dua induk agar hubungan lokal antar elemen tetap terjaga pada keturunan.
- Pada pembentukan anak pertama, proses persilangan pemetaan parsial dilakukan melalui langkah-langkah berikut:
  - Menentukan dua titik potong secara acak pada individu induk.
  - Segmen gen di antara kedua titik potong pada induk pertama disalin langsung ke anak pertama pada posisi yang sama.
  - Membangun peta pasangan gen antara segmen yang disalin dari induk pertama dan segmen yang bersesuaian pada induk kedua. Peta ini digunakan untuk menjaga konsistensi nilai gen.
  - Mengisi posisi yang kosong pada anak pertama dengan gen dari induk kedua:
    - Gen diambil dari induk kedua pada posisi yang sama
    - Jika gen tersebut sudah ada pada anak (karena ada dalam segmen yang disalin), maka gen tersebut diganti menggunakan peta pasangan sampai ditemukan gen yang belum ada dalam anak.
- Pembentukan anak kedua dilakukan dengan cara yang sama, dengan menukar peran induk pertama dan induk kedua.

```
Persilangan Pemetaan Parsial (Partially Mapped Crossover)
Catatan: indeks dimulai dari 0

Induk-induk hasil seleksi:
  induk 1: [9, 3, 6, 7, 4, 2, 1, 8, 5]
  induk 2: [4, 8, 5, 7, 9, 3, 2, 6, 1]

Penentuan segmen secara acak (indeks awal: = 1, indeks akhir: 5)
  Segmen induk 1: [3, 6, 7, 4, 2]
  Segmen induk 2: [8, 5, 7, 9, 3]

Pemetaan gen-gen pada segmen:
  Segmen induk 1 ke segmen induk 2 (pemetaan 1-2):
  {3: 8, 6: 5, 7: 7, 4: 9, 2: 3}
  Segmen induk 2 ke segmen induk 1 (pemetaan 2-1):
  {8: 3, 5: 6, 7: 7, 9: 4, 3: 2}

Pembentukan anak 1:
  Inisialisasi
    Anak 1:  [None, None, None, None, None, None, None, None, None]
  Salin segmen induk 1 ke anak 1 sesuai dengan posisinya:
    Anak 1:  [None, 3, 6, 7, 4, 2, None, None, None]
  Isi posisi yang kosong pada anak 1 dengan gen-gen dari induk 2:
   Indeks posisi: 0, gen pada induk 2: 4
    Gen: 4 dalam segmen induk 1, pemetaan [pemetaan 1-2] (gen: 4 -> gen: 9)
    Gen: 9 di luar segmen induk 1, isi dengan gen: 9
    Hasil sementara: Anak 1:  [9, 3, 6, 7, 4, 2, None, None, None]
   Indeks posisi: 6, gen pada induk 2: 2
    Gen: 2 dalam segmen induk 1, pemetaan [pemetaan 1-2] (gen: 2 -> gen: 3)
    Gen: 3 dalam segmen induk 1, pemetaan [pemetaan 1-2] (gen: 3 -> gen: 8)
    Gen: 8 tidak ada di segmen induk 1, isi dengan gen: 8
    Hasil sementara: Anak 1:  [9, 3, 6, 7, 4, 2, 8, None, None]
   Indeks posisi: 7, gen pada induk 2: 6
    Gen: 6 dalam segmen induk 1, pemetaan [pemetaan 1-2] (gen: 6 -> gen: 5)
    Gen: 5 di luar segmen induk 1, isi dengan gen: 5
    Hasil sementara: Anak 1:  [9, 3, 6, 7, 4, 2, 8, 5, None]
   Indeks posisi: 8, gen pada induk 2: 1
    Gen: 1 di luar segmen induk 1, isi dengan gen: 1
    Hasil sementara: Anak 1:  [9, 3, 6, 7, 4, 2, 8, 5, 1]

Hasil persilangan:
  Anak 1:  [9, 3, 6, 7, 4, 2, 8, 5, 1]
```

```
Persilangan Pemetaan Parsial (Partially Mapped Crossover)
Catatan: indeks dimulai dari 0

Induk-induk hasil seleksi:
  induk 1: [9, 3, 6, 7, 4, 2, 1, 8, 5]
  induk 2: [4, 8, 5, 7, 9, 3, 2, 6, 1]

Penentuan segmen secara acak (indeks awal: = 1, indeks akhir: 5)
  Segmen induk 1: [3, 6, 7, 4, 2]
  Segmen induk 2: [8, 5, 7, 9, 3]

Pemetaan gen-gen pada segmen:
  Segmen induk 1 ke segmen induk 2 (pemetaan 1-2):
  {3: 8, 6: 5, 7: 7, 4: 9, 2: 3}
  Segmen induk 2 ke segmen induk 1 (pemetaan 2-1):
  {8: 3, 5: 6, 7: 7, 9: 4, 3: 2}

Pembentukan anak 2:
  Inisialisasi
    Anak 2:  [None, None, None, None, None, None, None, None, None]
  Salin segmen induk 2 ke anak 2 sesuai dengan posisinya:
    Anak 2:  [None, 8, 5, 7, 9, 3, None, None, None]
  Isi posisi yang kosong pada anak 2 dengan gen-gen dari induk 1:
   Indeks posisi: 0, gen pada induk 1: 9
    Gen: 9 dalam segmen induk 2, pemetaan [pemetaan 2-1] (gen: 9 -> gen: 4)
    Gen: 4 di luar segmen induk 2, isi dengan gen: 4
    Hasil sementara: Anak 2:  [4, 8, 5, 7, 9, 3, None, None, None]
   Indeks posisi: 6, gen pada induk 1: 1
    Gen: 1 di luar segmen induk 2, isi dengan gen: 1
    Hasil sementara: Anak 2:  [4, 8, 5, 7, 9, 3, 1, None, None]
   Indeks posisi: 7, gen pada induk 1: 8
    Gen: 8 dalam segmen induk 2, pemetaan [pemetaan 2-1] (gen: 8 -> gen: 3)
    Gen: 3 dalam segmen induk 2, pemetaan [pemetaan 2-1] (gen: 3 -> gen: 2)
    Gen: 2 di luar segmen induk 2, isi dengan gen: 2
    Hasil sementara: Anak 2:  [4, 8, 5, 7, 9, 3, 1, 2, None]
   Indeks posisi: 8, gen pada induk 1: 5
    Gen: 5 dalam segmen induk 2, pemetaan [pemetaan 2-1] (gen: 5 -> gen: 6)
    Gen: 6 di luar segmen induk 2, isi dengan gen: 6
    Hasil sementara: Anak 2:  [4, 8, 5, 7, 9, 3, 1, 2, 6]

Hasil persilangan:
  Anak 2:  [4, 8, 5, 7, 9, 3, 1, 2, 6]
```

Beberapa Operator Persilangan untuk Representasi Bilangan Riil

- Persilangan Aritmatik (Arithmetic Crossover)
- Persilangan Campuran (Blend Crossover)
- Persilangan Biner Tersimulasi (Simulated Binary Crossover)

Persilangan Aritmatik

- Membentuk keturunan sebagai kombinasi linear dari gen-gen induk menggunakan suatu parameter acak α dengan nilai di antara 0 dan 1.
- Induk-induk: P_1 dan P_2
- Anak-anak:

$$C_1 = \alpha P_1 + (1 - \alpha) P_2$$
$$C_2 = (1 - \alpha) P_1 + \alpha P_2$$

```
Persilangan Aritmatik (Arithmatic Crossover)

Induk-induk hasil seleksi:
  Induk 1 = 125.4956
  Induk 2 = 157.6016

Pembangkitan nilai alpha secara acak (antara 0 dan 1):
  alpha = 0.4066

Hasil persilangan:
  Anak 1 = 144.5468
  Anak 2 = 138.5503
```

Persilangan Campuran

- Menghasilkan keturunan dari interval yang diperluas di sekitar nilai kedua
- Induk-induk: P_1 dan P_2

$$P_{\min} = \min\{P_1, P_2\}$$
$$P_{\max} = \max\{P_1, P_2\}$$

- Anak-anak:

$$C_1 = P_{\min} - \alpha(P_{\max} - P_{\min})$$
$$C_2 = P_{\max} + \alpha(P_{\max} - P_{\min})$$

Persilangan Campuran (Blending Crossover)
alpha = 0.30 (parameter > 0)

Induk-induk hasil seleksi:
  Induk 1 = 158.3591
  Induk 2 = 146.5754

Penentuan induk dengan nilai paling kecil dan paling besar:
  induk_min = 146.5754
  induk_max = 158.3591

Hasil persilangan:
  Anak 1 = 143.0403
  Anak 2 = 161.8943

Persilangan Biner Tersimulasi

- Operator persilangan yang paling populer untuk representasi bilangan riil
- Meniru perilaku operator persilangan satu-titik yang digunakan pada representasi biner, tetapi diterapkan pada individu dengan nilai kontinu.
- Induk-induk: P_1 dan P_2
- Anak-anak: C_1 dan C_2

$$C_1 = 0{,}5[(1+\beta)P_1 + (1-\beta)P_2]$$
$$C_1 = 0{,}5[(1-\beta)P_1 + (1+\beta)P_2]$$

- Parameter β:

$$\beta = \begin{cases} (2u)^{1/(\eta_c+1)} & ; u \leq 0{,}5 \\ \left(1/\left(2(1-u)\right)\right)^{1/(\eta_c+1)} & ; u > 0{,}5 \end{cases}$$

- u → bilangan random
- η_c → indeks distribusi persilangan

```
Persilangan Biner Tersimulasi (Simulated Binary Crossover)
eta_c = 5 (parameter)

Induk-induk hasil seleksi:
  Induk 1 = 63.6493
  Induk 2 = 178.4372

Pembangkitan bilangan acak (distribusi seragam [0, 1]):
  bilangan acak, u = 0.5235
Penentuan nilai beta:
  beta = 1.0081

Hasil persilangan:
  Anak 1 = 153.8429
  Anak 2 = 89.6809
```

Mutasi

- Proses perubahan pada individu dengan cara mengganti satu atau beberapa gen di dalamnya secara acak.
- Terjadinya mutasi dikendalikan oleh suatu parameter yang disebut tingkat mutasi (mutation rate), yang menentukan seberapa besar probabilitas terjadinya perubahan gen pada setiap individu.
- Mempertahankan keragaman genetik dalam populasi serta mencegah algoritma terjebak pada solusi lokal.
- Dengan adanya mutasi, populasi dapat mengeksplorasi area pencarian yang lebih luas, sehingga meningkatkan probabilitas untuk menemukan solusi yang lebih baik.
- Mengacu pada analogi seleksi alam, tingkat mutasi umumnya ditetapkan relatif kecil agar keberagaman solusi tetap terjaga tanpa mengganggu kestabilan populasi yang telah terbentuk.

Beberapa Operator Mutasi untuk Representasi Kode Biner

- Mutasi Pembalikan Bit (Bit Flip Mutation)
- Mutasi Seragam (Uniform Mutation)

Operator Pembalikan Bit

- Mutasi paling sederhana dan paling banyak digunakan untuk representasi biner.
- Memilih satu posisi bit secara acak, kemudian membalikkan nilainya, jika 0 menjadi 1, dan jika 1 menjadi 0.
- Memperkenalkan variasi baru tanpa mengubah struktur individu secara signifikan.

```
Mutasi Pembalikan Bit (Bit Flip Mutation)
Catatan: indeks mulai dari 0

Individu sebelum mutasi: [0, 0, 1, 0, 1, 0, 0, 1, 0, 1]
Proses mutasi:
  Pilih secara acak indeks dari bit yang nilainya akan dibalik = 7
  Balik nilai bit pada indeks posisi = 7 (1 -> 0)
Individu setelah mutasi: [0, 0, 1, 0, 1, 0, 0, 0, 0, 1]
```

Mutasi Seragam

- Setiap bit dalam individu memiliki probabilitas yang sama untuk mengalami pembalikan nilai.
- Lebih dari satu bit dapat mengalami mutasi dalam satu proses.
- Meningkatkan variasi secara merata di seluruh individu.

```
Mutasi Seragam (Uniform Mutation)
Catatan: indeks mulai dari 0

Individu sebelum mutasi: [0, 1, 1, 0, 0, 1, 1, 0, 0, 1]
Proses mutasi
  Pilih secara acak indeks-indeks dari bit yang nilainya akan dibalik
  -> Indeks: [1, 3, 5, 6, 8]
  Balik nilai bit pada indeks posisi terpilih
    Indeks = 1 (1 -> 0)
    Indeks = 3 (0 -> 1)
    Indeks = 5 (1 -> 0)
    Indeks = 6 (1 -> 0)
    Indeks = 8 (0 -> 1)
Individu setelah mutasi: [0, 0, 1, 1, 0, 0, 0, 0, 1, 1]
```

Beberapa Operator Mutasi untuk Representasi Permutasi

- Mutasi Pertukaran (Swap Mutation)
- Mutasi Penyisipan (Insertion Mutation)
- Mutasi Pembalikan (Inversion Mutation)

Mutasi Pertukaran

- Mutasi yang umum digunakan pada individu yang direpresentasikan dalam bentuk permutasi.
- Memilih dua posisi gen secara acak dalam individu, kemudian menukar nilai gen pada kedua posisi tersebut.
- Urutan gen berubah tetapi tidak ada gen yang hilang atau terduplikasi, sehingga sifat permutasi tetap terjaga.
- Memperkenalkan variasi lokal dalam urutan individu, serta meningkatkan eksplorasi ruang solusi tanpa merusak struktur dasar solusi yang sudah baik.

```
Mutasi Pertukaran (Swap Mutation)
Catatan: indeks mulai dari 0

Individu sebelum mutasi: [4, 6, 8, 7, 5, 1, 3, 9, 2]
Proses mutasi:
  Pilih secara acak indeks gen pertama yang akan dipertukarkan = 8
  Pilih secara acak indeks gen kedua yang akan dipertukarkan = 5
  Pertukarkan gen-gen pada:
    indeks = 8 (gen = 2) dengan gen pada indeks = 5 (gen = 1)
Individu setelah mutasi: [4, 6, 8, 7, 5, 2, 3, 9, 1]
```

Mutasi Penyisipan

- Mengubah individu dengan cara memindahkan satu gen dari posisinya semula dan menyisipkannya ke posisi lain di dalam individu.
- Meningkatkan keragaman populasi, mencegah algoritma terjebak pada solusi lokal, dan memberikan probabilitas bagi pembentukan kombinasi gen baru yang lebih baik

```
Mutasi Penyisipan (Insertion Mutation)
Catatan: indeks mulai dari 0

Individu sebelum mutasi: [1, 2, 8, 6, 5, 9, 4, 3, 7]
Proses mutasi:
  Pilih secara acak indeks gen yang akan disisipkan = 6
  Pilih secara acak indeks tujuan dari gen yang disisipkan = 2
  Hapus gen pada indeks = 6 (gen = 4) -> [1, 2, 8, 6, 5, 9, 3, 7]
  Sisipkan gen = 4 pada indeks = 2 dari individu hasil penghapusan
  -> [1, 2, 4, 8, 6, 5, 9, 3, 7]
Individu setelah mutasi: [1, 2, 4, 8, 6, 5, 9, 3, 7]
```

Mutasi Pembalikan

- Membalik urutan gen dalam suatu segmen individu.
- Dua posisi acak pada individu dipilih untuk menentukan batas awal dan akhir segmen yang akan dibalik.
- Mengubah struktur urutan gen tanpa mengubah isi gen, sehingga dapat mengeksplorasi kombinasi baru yang mungkin lebih baik tanpa merusak informasi genetik yang sudah ada.

```
Mutasi Pembalikan (Inversion Mutation)
Catatan: indeks mulai dari 0

Individu sebelum mutasi: [1, 9, 7, 4, 3, 2, 5, 8, 6]
Proses mutasi:
  Pilih secara acak indeks awal segmen yang urutan gennya akan dibalik = 2
  Pilih secara acak indeks akhir segmen yang urutan gennya akan dibalik = 6
  -> Segmen yang akan dibalik urutan gennya: [7, 4, 3, 2, 5]
  Balik urutan gen pada segmen -> [5, 2, 3, 4, 7]
Individu setelah mutasi: [1, 9, 5, 2, 3, 4, 7, 8, 6]
```

Beberapa Operator untuk Representasi Bilangan Riil

- Mutasi Seragam (Uniform Mutation)
- Mutasi Gaussian (Gaussian Mutation)
- Mutasi Polinomial (Polynomial Mutation)

Mutasi Seragam

- Mengganti nilai gen yang dipilih secara acak dengan nilai baru yang dihasilkan secara acak dari interval pencarian $[x_{\min}, x_{\max}]$
- Kelebihan:
  - Kesederhanaan dan meningkatkan eksplorasi global
- Kelemahan:
  - Terlalu acak jika diterapkan pada tahap akhir evolusi
- Rumus: $x' = \text{Seragam Kontinu}(x_{\min}, x_{\max})$

```
Mutasi Seragam (Uniform Mutation)
batas bawah dari nilai x = 0
batas atas dari nilai x  = 200

Individu sebelum mutasi: 61.6570
Proses mutasi:
  Bangkitkan variat acak seragam kontinu antara batas bawah dan batas atas
    variat = 162.8077
  Ganti individu sebelum mutasi dengan variat
    hasil = 162.8077
Individu setelah mutasi: 162.8077
```

Mutasi Gaussian

- Nilai gen diubah dengan penambahan gangguan acak yang mengikuti distribusi normal (Gaussian) dengan rata-rata 0 dan simpangan baku tertentu σ
- Besaran σ mengontrol besarnya perubahan.
  - Jika nilainya kecil menyebabkan mutasi menjadi halus (eksploitasi).
  - Jika nilainya besar, mutasi menjadi luas (eksplorasi).
- Rumus: $x' = x + \text{Gaussian}(0, \sigma)$

```
Mutasi Gaussian (Gaussian Mutation)
sigma = 2.00 (parameter > 0)

Individu sebelum mutasi: 32.4747
Proses mutasi:
  Bangkitkan variat acak Gaussian/normal (rataan = 0 dan sigma)
    variat = -1.0830
  Lakukan operasi:
    (Individu setelah mutasi)  = (individu sebelum mutasi) + (variat)
    hasil = 31.3917
Individu setelah mutasi: 31.3917
```

Mutasi Polinomial

- Mutasi yang digunakan luas dalam algoritma genetika berbasis bilangan riil.
- Mutasi dilakukan dengan menghasilkan perubahan kecil berdasarkan indeks distribusi mutasi (mutation distribution index)

Diberikan gen i dari individu yang akan dimutasi adalah x_i. Gen i dari individu hasil mutasi ditentukan dengan:

$$x_i' = x_i + \delta_i \left(x_{\max} - x_{\min} \right)$$

dengan

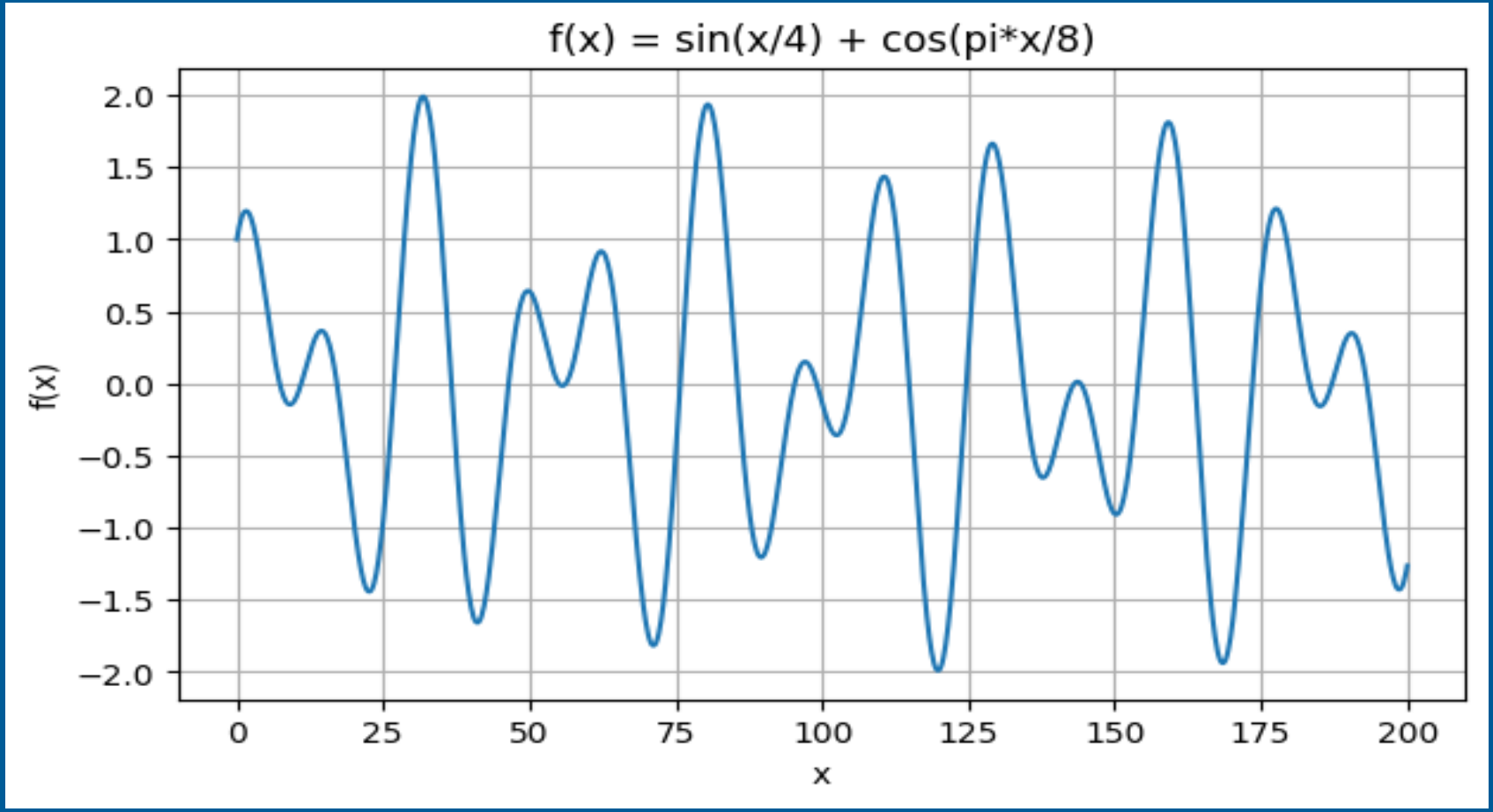
$$\delta_i = \begin{cases} \left(2u\right)^{1/(\eta_m+1)} - 1 & ;u \le 0,5 \\ 1 - \left(2\left(1-u\right)\right)^{1/(\eta_m+1)} & ;u > 0,5 \end{cases}$$

Besaran u merupakan bilangan acak antara 0 dan 1. Parameter η_m mengendalikan lebar distribusi keturunan. Jika η_m besar, maka mutasi yang dilakukan kecil (berfokus pada eksploitasi). Jika η_m besar, maka mutasi yang dilakukan besar (berfokus pada eksplorasi).

Penerapan GA untuk Masalah Optimisasi Numerik

Memaksimumkan $f\left(x\right)=\sin\left(\frac{x}{4}\right)+\cos\left(\frac{\pi x}{8}\right)$

dengan kendala: $0\le x\le 200$ dan bilangan bulat



Fitur-Fitur dan Parameter-Parameter GA

- Fitur-fitur:
  - Representasi Individu: Representasi Kode Biner
  - Metode Seleksi: Roda Rolet
  - Operator Persilangan: Persilangan Satu-titik
  - Operator Mutasi: Mutasi Pembalikan Bit
- Nilai-nilai Parameter:
  - Ukuran Populasi = 10
  - Tingkat Persilangan = 0,8
  - Tingkat Mutasi = 0,1
  - Jumlah Generasi Maksimum = 100

```
Generasi 0
Populasi Awal:

Individu 1:    [1, 0, 1, 0, 0, 1, 0, 0]
Individu 2:    [0, 1, 1, 0, 0, 1, 1, 0]
Individu 3:    [1, 1, 0, 0, 1, 1, 1, 1]
Individu 4:    [0, 1, 0, 1, 1, 1, 0, 1]
Individu 5:    [1, 1, 0, 0, 0, 1, 0, 1]
Individu 6:    [1, 1, 1, 0, 0, 1, 0, 1]
Individu 7:    [0, 0, 1, 0, 1, 1, 0, 1]
Individu 8:    [1, 1, 1, 1, 1, 0, 0, 0]
Individu 9:    [0, 1, 1, 1, 0, 0, 1, 1]
Individu 10:   [1, 0, 1, 0, 0, 1, 1, 0]
```

Hasil Dekoding, Nilai Fungsi Tujuan, dan Nilai Fitnes
Untuk solusi yang taklayak, x < 0 atau x > 200, tambahkan penalti (-100) pada nilai fungsi tujuan
Nilai fitness = Nilai fungsi tujuan + 1000 (agar nilai fitness > 0)

Individu 1: [1, 0, 1, 0, 0, 1, 0, 0], x = 164, Nilai fungsi tujuan = -0.1586, Nilai fitnes = 999.8414
Individu 2: [0, 1, 1, 0, 0, 1, 1, 0], x = 102, Nilai fungsi tujuan = -0.3480, Nilai fitnes = 999.6520
Individu 3: [1, 1, 0, 0, 1, 1, 1, 1], x = 207, Nilai fungsi tujuan = -98.0798, Nilai fitnes = 901.9202
Individu 4: [0, 1, 0, 1, 1, 1, 0, 1], x = 93, Nilai fungsi tujuan = -0.5691, Nilai fitnes = 999.4309
Individu 5: [1, 1, 0, 0, 0, 1, 0, 1], x = 197, Nilai fungsi tujuan = -1.2324, Nilai fitnes = 998.7676
Individu 6: [1, 1, 1, 0, 0, 1, 0, 1], x = 229, Nilai fungsi tujuan = -99.7374, Nilai fitnes = 900.2626
Individu 7: [0, 0, 1, 0, 1, 1, 0, 1], x = 45, Nilai fungsi tujuan = -0.5851, Nilai fitnes = 999.4149
Individu 8: [1, 1, 1, 1, 1, 0, 0, 0], x = 248, Nilai fungsi tujuan = -101.7392, Nilai fitnes = 898.2608
Individu 9: [0, 1, 1, 1, 0, 0, 1, 1], x = 115, Nilai fungsi tujuan = -0.0752, Nilai fitnes = 999.9248
Individu 10: [1, 0, 1, 0, 0, 1, 1, 0], x = 166, Nilai fungsi tujuan = -1.3197, Nilai fitnes = 998.6803

Inisialisasi solusi terbaik saat ini:
x = 115, Nilai fungsi tujuan = -0.0752, Nilai fitnes = 999.9248

Probabilitas dan Probabilitas Kumulatif
(Digunakan untuk metode seleksi roda rolet)

Individu 1: [1, 0, 1, 0, 0, 1, 0, 0], Probabilitas = 0.1031, Probabilitas Kumulatif = 0.1031
Individu 2: [0, 1, 1, 0, 0, 1, 1, 0], Probabilitas = 0.1031, Probabilitas Kumulatif = 0.2062
Individu 3: [1, 1, 0, 0, 1, 1, 1, 1], Probabilitas = 0.0930, Probabilitas Kumulatif = 0.2992
Individu 4: [0, 1, 0, 1, 1, 1, 0, 1], Probabilitas = 0.1031, Probabilitas Kumulatif = 0.4023
Individu 5: [1, 1, 0, 0, 0, 1, 0, 1], Probabilitas = 0.1030, Probabilitas Kumulatif = 0.5053
Individu 6: [1, 1, 1, 0, 0, 1, 0, 1], Probabilitas = 0.0928, Probabilitas Kumulatif = 0.5982
Individu 7: [0, 0, 1, 0, 1, 1, 0, 1], Probabilitas = 0.1031, Probabilitas Kumulatif = 0.7012
Individu 8: [1, 1, 1, 1, 1, 0, 0, 0], Probabilitas = 0.0926, Probabilitas Kumulatif = 0.7939
Individu 9: [0, 1, 1, 1, 0, 0, 1, 1], Probabilitas = 0.1031, Probabilitas Kumulatif = 0.8970
Individu 10: [1, 0, 1, 0, 0, 1, 1, 0], Probabilitas = 0.1030, Probabilitas Kumulatif = 1.0000

Seleksi = 1
Bilangan random = 0.8199 -> Individu terpilih: 9 -> Induk 1 = [0, 1, 1, 1, 0, 0, 1, 1]
Bilangan random = 0.0676 -> Individu terpilih: 1 -> Induk 2 = [1, 0, 1, 0, 0, 1, 0, 0]

Persilangan Induk 1 dan Induk 2
Bilangan random = 0.9438 > tingkat persilangan = 0.8 -> persilangan tidak dilakukan
Anak 1 disalin langsung dari Induk 1, Anak 2 disalin langsung dari Induk 2
Anak 1 = [0, 1, 1, 1, 0, 0, 1, 1]
Anak 2 = [1, 0, 1, 0, 0, 1, 0, 0]

Mutasi pada Anak 1
Bilangan random = 0.6514 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 1 = [0, 1, 1, 1, 0, 0, 1, 1]

Mutasi pada Anak 2
Bilangan random = 0.2335 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 2 = [1, 0, 1, 0, 0, 1, 0, 0]

Seleksi = 2
Bilangan random = 0.8296 -> Individu terpilih: 9 -> Induk 1 = [0, 1, 1, 1, 0, 0, 1, 1]
Bilangan random = 0.9852 -> Individu terpilih: 10 -> Induk 2 = [1, 0, 1, 0, 0, 1, 1, 0]

Persilangan Induk 1 dan Induk 2
Bilangan random = 0.4921 <= tingkat persilangan = 0.8 -> persilangan dilakukan
Operator persilangan: Persilangan satu-titik
Titik potong = 4
Anak 1 = [0, 1, 1, 1, 0, 1, 1, 0]
Anak 2 = [1, 0, 1, 0, 0, 0, 1, 1]

Mutasi pada Anak 1
Bilangan random = 0.2153 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 1 = [0, 1, 1, 1, 0, 1, 1, 0]

Mutasi pada Anak 2
Bilangan random = 0.3949 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 2 = [1, 0, 1, 0, 0, 0, 1, 1]

Seleksi = 3
Bilangan random = 0.7180 -> Individu terpilih: 8 -> Induk 1 = [1, 1, 1, 1, 1, 0, 0, 0]
Bilangan random = 0.4185 -> Individu terpilih: 5 -> Induk 2 = [1, 1, 0, 0, 0, 1, 0, 1]

Persilangan Induk 1 dan Induk 2
Bilangan random = 0.5694 <= tingkat persilangan = 0.8 -> persilangan dilakukan
Operator persilangan: Persilangan satu-titik
Titik potong = 4
Anak 1 = [1, 1, 1, 1, 0, 1, 0, 1]
Anak 2 = [1, 1, 0, 0, 1, 0, 0, 0]

Mutasi pada Anak 1
Bilangan random = 0.0657 <= tingkat mutasi = 0.1 -> mutasi dilakukan
Operator mutasi: Mutasi Pembalikan Bit
Indeks gen yang dibalik = 4
Anak 1 = [1, 1, 1, 1, 1, 1, 0, 1]

Mutasi pada Anak 2
Bilangan random = 0.2314 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 2 = [1, 1, 0, 0, 1, 0, 0, 0]

Seleksi = 4

Bilangan random = 0.6510 -> Individu terpilih: 7 -> Induk 1 = [0, 0, 1, 0, 1, 1, 0, 1]
Bilangan random = 0.7488 -> Individu terpilih: 8 -> Induk 2 = [1, 1, 1, 1, 1, 0, 0, 0]

Persilangan Induk 1 dan Induk 2
Bilangan random = 0.1351 <= tingkat persilangan = 0.8 -> persilangan dilakukan
Operator persilangan: Persilangan satu-titik
Titik potong = 6
Anak 1 = [0, 0, 1, 0, 1, 1, 0, 0]
Anak 2 = [1, 1, 1, 1, 1, 0, 0, 1]

Mutasi pada Anak 1
Bilangan random = 0.7946 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 1 = [0, 0, 1, 0, 1, 1, 0, 0]

Mutasi pada Anak 2
Bilangan random = 0.4775 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 2 = [1, 1, 1, 1, 1, 0, 0, 1]

Seleksi = 5

Bilangan random = 0.2552 -> Individu terpilih: 3 -> Induk 1 = [1, 1, 0, 0, 1, 1, 1, 1]
Bilangan random = 0.8932 -> Individu terpilih: 9 -> Induk 2 = [0, 1, 1, 1, 0, 0, 1, 1]
Persilangan Induk 1 dan Induk 2
Bilangan random = 0.9066 > tingkat persilangan = 0.8 -> persilangan tidak dilakukan

Anak 1 disalin langsung dari Induk 1, Anak 2 disalin langsung dari Induk 2
Anak 1 = [1, 1, 0, 0, 1, 1, 1, 1]
Anak 2 = [0, 1, 1, 1, 0, 0, 1, 1]

Mutasi pada Anak 1
Bilangan random = 0.1729 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 1 = [1, 1, 0, 0, 1, 1, 1, 1]

Mutasi pada Anak 2
Bilangan random = 0.9097 > tingkat mutasi = 0.1 -> mutasi tidak dilakukan
Anak 2 = [0, 1, 1, 1, 0, 0, 1, 1]

```
Generasi 1
Populasi Baru

Individu 1:    [0, 1, 1, 1, 0, 0, 1, 1]
Individu 2:    [1, 0, 1, 0, 0, 1, 0, 0]
Individu 3:    [0, 1, 1, 1, 0, 1, 1, 0]
Individu 4:    [1, 0, 1, 0, 0, 0, 1, 1]
Individu 5:    [1, 1, 1, 1, 1, 1, 0, 1]
Individu 6:    [1, 1, 0, 0, 1, 0, 0, 0]
Individu 7:    [0, 0, 1, 0, 1, 1, 0, 0]
Individu 8:    [1, 1, 1, 1, 1, 0, 0, 1]
Individu 9:    [1, 1, 0, 0, 1, 1, 1, 1]
Individu 10:   [0, 1, 1, 1, 0, 0, 1, 1]
```

```
Hasil Dekoding, Nilai Fungsi Tujuan, dan Nilai Fitnes
Untuk solusi yang taklayak, x < 0 atau x > 200, tambahkan penalti (-100) pada nilai fungsi tujuan
Nilai fitness = Nilai fungsi tujuan + 1000 (agar nilai fitness > 0)


Individu 1:  [0, 1, 1, 1, 0, 0, 1, 1], x = 115, Nilai fungsi tujuan = -0.0752,   Nilai fitnes = 999.9248
Individu 2:  [1, 0, 1, 0, 0, 1, 0, 0], x = 164, Nilai fungsi tujuan = -0.1586,   Nilai fitnes = 999.8414
Individu 3:  [0, 1, 1, 1, 0, 1, 1, 0], x = 118, Nilai fungsi tujuan = -1.6481,   Nilai fitnes = 998.3519
Individu 4:  [1, 0, 1, 0, 0, 0, 1, 1], x = 163, Nilai fungsi tujuan = 0.4733,    Nilai fitnes = 1000.4733
Individu 5:  [1, 1, 1, 1, 1, 1, 0, 1], x = 253, Nilai fungsi tujuan = -99.2112,  Nilai fitnes = 900.7888
Individu 6:  [1, 1, 0, 0, 1, 0, 0, 0], x = 200, Nilai fungsi tujuan = -1.2624,   Nilai fitnes = 998.7376
Individu 7:  [0, 0, 1, 0, 1, 1, 0, 0], x = 44,  Nilai fungsi tujuan = -1.0000,   Nilai fitnes = 999.0000
Individu 8:  [1, 1, 1, 1, 1, 0, 0, 1], x = 249, Nilai fungsi tujuan = -101.4735, Nilai fitnes = 898.5265
Individu 9:  [1, 1, 0, 0, 1, 1, 1, 1], x = 207, Nilai fungsi tujuan = -98.0798,  Nilai fitnes = 901.9202
Individu 10: [0, 1, 1, 1, 0, 0, 1, 1], x = 115, Nilai fungsi tujuan = -0.0752,   Nilai fitnes = 999.9248

Solusi terbaik pada generasi ini:
x = 163, Nilai fungsi tujuan = 0.4733, Nilai fitness = 1000.4733
Nilai fitness lebih baik dari solusi terbaik saat ini

Solusi terbaik saat ini --> perbarui:
x = 163, Nilai fungsi tujuan = 0.4733, Nilai fitness = 1000.4733
```

Prosedur dilanjutkan hingga kriteria henti tercapai
(dalam hal ini hingga, misal 100 generasi)

Solusi terbaik hingga 100 generasi:
x = 32, nilai fungsi tujuan = 1.9894, nilai fitness = 1001.9894

Penerapan Genetic Algorithm untuk Masalah Penjual Keliling

Tabel Jarak

Dari simpul \ Ke simpul	0	1	2	3	4	5	6	7	8	9
0	0	9	14	10	20	11	15	17	8	3
1	8	0	17	4	13	8	5	1	5	14
2	8	17	0	16	10	14	3	8	4	5
3	9	19	1	0	8	4	19	6	14	12
4	10	5	19	10	0	15	12	19	17	7
5	16	19	1	17	2	0	2	3	18	10
6	13	17	3	18	8	18	0	4	17	5
7	8	6	4	16	11	6	17	0	4	11
8	17	15	20	2	2	7	12	8	0	17
9	17	2	6	8	20	10	6	15	4	0

Fitur-Fitur dan Parameter-Parameter GA

- Fitur-fitur:
  - Representasi Individu: Representasi Permutasi
  - Metode Seleksi: Roda Rolet
  - Operator Persilangan: Persilangan Berbasis Posisi
  - Operator Mutasi: Mutasi Pertukaran
- Nilai-nilai Parameter:
  - Ukuran Populasi = 10
  - Tingkat Persilangan = 0,8
  - Tingkat Mutasi = 0,1
  - Jumlah Generasi Maksimum = 100

```
Generasi = 0
Populasi Awal:

Individu 1:    [5, 4, 3, 6, 1, 9, 7, 8, 2]
Individu 2:    [1, 4, 6, 3, 2, 7, 9, 8, 5]
Individu 3:    [1, 5, 2, 7, 6, 4, 8, 3, 9]
Individu 4:    [4, 3, 5, 1, 9, 8, 6, 7, 2]
Individu 5:    [8, 4, 1, 6, 2, 5, 3, 9, 7]
Individu 6:    [9, 6, 8, 1, 7, 5, 2, 3, 4]
Individu 7:    [5, 6, 2, 1, 7, 3, 4, 9, 8]
Individu 8:    [6, 9, 4, 5, 7, 8, 1, 2, 3]
Individu 9:    [1, 7, 4, 2, 9, 5, 3, 8, 6]
Individu 10:   [3, 7, 1, 4, 8, 2, 5, 6, 9]
```

Solusi, Nilai fungsi tujuan (jarak total), dan Nilai fitnes:
Nilai fitness = 1/Jarak total

Individu 1: Solusi: [0, 5, 4, 3, 6, 1, 9, 7, 8, 2, 0], Jarak total = 121, Nilai fitnes = 0.0083
Individu 2: Solusi: [0, 1, 4, 6, 3, 2, 7, 9, 8, 5, 0], Jarak total = 99, Nilai fitnes = 0.0101
Individu 3: Solusi: [0, 1, 5, 2, 7, 6, 4, 8, 3, 9, 0], Jarak total = 99, Nilai fitnes = 0.0101
Individu 4: Solusi: [0, 4, 3, 5, 1, 9, 8, 6, 7, 2, 0], Jarak total = 100, Nilai fitnes = 0.0100
Individu 5: Solusi: [0, 8, 4, 1, 6, 2, 5, 3, 9, 7, 0], Jarak total = 89, Nilai fitnes = 0.0112
Individu 6: Solusi: [0, 9, 6, 8, 1, 7, 5, 2, 3, 4, 0], Jarak total = 83, Nilai fitnes = 0.0120
Individu 7: Solusi: [0, 5, 6, 2, 1, 7, 3, 4, 9, 8, 0], Jarak total = 86, Nilai fitnes = 0.0116
Individu 8: Solusi: [0, 6, 9, 4, 5, 7, 8, 1, 2, 3, 0], Jarak total = 119, Nilai fitnes = 0.0084
Individu 9: Solusi: [0, 1, 7, 4, 2, 9, 5, 3, 8, 6, 0], Jarak total = 111, Nilai fitnes = 0.0090
Individu 10: Solusi: [0, 3, 7, 1, 4, 8, 2, 5, 6, 9, 0], Jarak total = 110, Nilai fitnes = 0.0091

Inisialisasi solusi terbaik saat ini:
Solusi: [0, 9, 6, 8, 1, 7, 5, 2, 3, 4, 0], Jarak total = 83, Nilai fitnes = 0.0120

```
Probabilitas dan probabilitas kumulatif:
(Digunakan untuk metode seleksi roda rolet)

Individu 1:       [5, 4, 3, 6, 1, 9, 7, 8, 2], Probabilitas = 0.0827, Probabilitas kumulatif = 0.0827
Individu 2:       [1, 4, 6, 3, 2, 7, 9, 8, 5], Probabilitas = 0.1011, Probabilitas kumulatif = 0.1839
Individu 3:       [1, 5, 2, 7, 6, 4, 8, 3, 9], Probabilitas = 0.1011, Probabilitas kumulatif = 0.2850
Individu 4:       [4, 3, 5, 1, 9, 8, 6, 7, 2], Probabilitas = 0.1001, Probabilitas kumulatif = 0.3851
Individu 5:       [8, 4, 1, 6, 2, 5, 3, 9, 7], Probabilitas = 0.1125, Probabilitas kumulatif = 0.4976
Individu 6:       [9, 6, 8, 1, 7, 5, 2, 3, 4], Probabilitas = 0.1206, Probabilitas kumulatif = 0.6182
Individu 7:       [5, 6, 2, 1, 7, 3, 4, 9, 8], Probabilitas = 0.1164, Probabilitas kumulatif = 0.7347
Individu 8:       [6, 9, 4, 5, 7, 8, 1, 2, 3], Probabilitas = 0.0841, Probabilitas kumulatif = 0.8188
Individu 9:       [1, 7, 4, 2, 9, 5, 3, 8, 6], Probabilitas = 0.0902, Probabilitas kumulatif = 0.9090
Individu 10:      [3, 7, 1, 4, 8, 2, 5, 6, 9], Probabilitas = 0.0910, Probabilitas kumulatif = 1.0000
```

```
Seleksi: 1

Pemilihan kromosom-kromosom sebagai induk:
  Bilangan random: 0.9226, individu terpilih 10 -> induk 1: [3, 7, 1, 4, 8, 2, 5, 6, 9]
  Bilangan random: 0.8501, individu terpilih 9  -> induk 2: [1, 7, 4, 2, 9, 5, 3, 8, 6]

Persilangan
  Bilangan random: 0.2314 <= tingkat persilangan (0.8) --> persilangan dilakukan
  Operator Persilangan: Persilangan berbasis posisi
  Indeks posisi yang dipilih: [0, 2, 8]
  Pembentukan anak 1:
  - Indeks dari gen yang disalin dari induk 1 ke anak 1: [0, 2, 8], gen: [3, 1, 9]
  - Indeks yang lain diisi dari induk 2 secara berurutan dengan melewatkan gen yang sudah muncul
  Anak 1:  [3, 7, 1, 4, 2, 5, 8, 6, 9]
  Pembentukan anak 1:
  - Indeks dari gen yang disalin dari induk 2 ke anak 2: [0, 2, 8], gen: [1, 4, 6]
  - Indeks yang lain diisi dari induk 1 secara berurutan dengan melewatkan gen yang sudah muncul
  Anak 2:  [1, 3, 4, 7, 8, 2, 5, 9, 6]

Mutasi pada anak 1
Bilangan random: 0.0657 <= tingkat mutasi (0.1) --> mutasi dilakukan
Operator mutasi: Mutasi pertukaran
Anak 1 (sebelum mutasi):  [3, 7, 1, 4, 2, 5, 8, 6, 9]
Indeks posisi gen yang dipertukarkan: 2 dan 8
Anak 1 (sesudah mutasi): [3, 7, 9, 4, 2, 5, 8, 6, 1]

Mutasi pada anak 2
Bilangan random: 0.5027 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 2: [1, 3, 4, 7, 8, 2, 5, 9, 6]
```

```
Seleksi: 2

Pemilihan kromosom-kromosom sebagai induk:
  Bilangan random: 0.0594, individu terpilih 1 -> induk 1: [5, 4, 3, 6, 1, 9, 7, 8, 2]
  Bilangan random: 0.2256, individu terpilih 3 -> induk 2: [1, 5, 2, 7, 6, 4, 8, 3, 9]

Persilangan
  Bilangan random: 0.9573 > tingkat persilangan (0.8) --> persilangan tidak dilakukan
  Salin langsung induk 1 dan induk 2 masing-masing ke anak 1 dan anak 2
  Anak 1:  [5, 4, 3, 6, 1, 9, 7, 8, 2]
  Anak 2:  [1, 5, 2, 7, 6, 4, 8, 3, 9]

Mutasi pada anak 1
Bilangan random: 0.1647 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 1: [5, 4, 3, 6, 1, 9, 7, 8, 2]

Mutasi pada anak 2
Bilangan random: 0.3561 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 2: [1, 5, 2, 7, 6, 4, 8, 3, 9]
```

```
Seleksi: 3

Pemilihan kromosom-kromosom sebagai induk:
  Bilangan random: 0.9768, individu terpilih 10 -> induk 1: [3, 7, 1, 4, 8, 2, 5, 6, 9]
  Bilangan random: 0.4514, individu terpilih 5  -> induk 2: [8, 4, 1, 6, 2, 5, 3, 9, 7]

Persilangan
  Bilangan random: 0.6672 <= tingkat persilangan (0.8) --> persilangan dilakukan
  Operator Persilangan: Persilangan berbasis posisi
  Indeks posisi yang dipilih: [0, 1, 2, 3, 4, 6, 8]
  Pembentukan anak 1:
  - Indeks dari gen yang disalin dari induk 1 ke anak 1: [0, 1, 2, 3, 4, 6, 8], gen: [3, 7, 1, 4, 8, 5, 9]
  - Indeks yang lain diisi dari induk 2 secara berurutan dengan melewatkan gen yang sudah muncul
  Anak 1:  [3, 7, 1, 4, 8, 6, 5, 2, 9]
  Pembentukan anak 1:
  - Indeks dari gen yang disalin dari induk 2 ke anak 2: [0, 1, 2, 3, 4, 6, 8], gen: [8, 4, 1, 6, 2, 3, 7]
  - Indeks yang lain diisi dari induk 1 secara berurutan dengan melewatkan gen yang sudah muncul
  Anak 2:  [8, 4, 1, 6, 2, 5, 3, 9, 7]

Mutasi pada anak 1
Bilangan random: 0.3926 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 1: [3, 7, 1, 4, 8, 6, 5, 2, 9]

Mutasi pada anak 2
Bilangan random: 0.6749 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 2: [8, 4, 1, 6, 2, 5, 3, 9, 7]
```

```
Seleksi: 4

Pemilihan kromosom-kromosom sebagai induk:
  Bilangan random: 0.4887, individu terpilih 5 -> induk 1: [8, 4, 1, 6, 2, 5, 3, 9, 7]
  Bilangan random: 0.5244, individu terpilih 6 -> induk 2: [9, 6, 8, 1, 7, 5, 2, 3, 4]

Persilangan
  Bilangan random: 0.3313 <= tingkat persilangan (0.8) --> persilangan dilakukan
  Operator Persilangan: Persilangan berbasis posisi
  Indeks posisi yang dipilih: [0, 2, 3, 8]
  Pembentukan anak 1:
  - Indeks dari gen yang disalin dari induk 1 ke anak 1: [0, 2, 3, 8], gen: [8, 1, 6, 7]
  - Indeks yang lain diisi dari induk 2 secara berurutan dengan melewatkan gen yang sudah muncul
  Anak 1:  [8, 9, 1, 6, 5, 2, 3, 4, 7]
  Pembentukan anak 1:
  - Indeks dari gen yang disalin dari induk 2 ke anak 2: [0, 2, 3, 8], gen: [9, 8, 1, 4]
  - Indeks yang lain diisi dari induk 1 secara berurutan dengan melewatkan gen yang sudah muncul
  Anak 2:  [9, 6, 8, 1, 2, 5, 3, 7, 4]

Mutasi pada anak 1
Bilangan random: 0.2504 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 1: [8, 9, 1, 6, 5, 2, 3, 4, 7]

Mutasi pada anak 2
Bilangan random: 0.1584 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 2: [9, 6, 8, 1, 2, 5, 3, 7, 4]
```

```
Seleksi: 5

Pemilihan kromosom-kromosom sebagai induk:
  Bilangan random: 0.5404, individu terpilih 6 -> induk 1: [9, 6, 8, 1, 7, 5, 2, 3, 4]
  Bilangan random: 0.1486, individu terpilih 2 -> induk 2: [1, 4, 6, 3, 2, 7, 9, 8, 5]

Persilangan
  Bilangan random: 0.6099 <= tingkat persilangan (0.8) --> persilangan dilakukan
  Operator Persilangan: Persilangan berbasis posisi
  Indeks posisi yang dipilih: [4, 6, 7]
  Pembentukan anak 1:
  - Indeks dari gen yang disalin dari induk 1 ke anak 1: [4, 6, 7], gen: [7, 2, 3]
  - Indeks yang lain diisi dari induk 2 secara berurutan dengan melewatkan gen yang sudah muncul
  Anak 1:  [1, 4, 6, 9, 7, 8, 2, 3, 5]
  Pembentukan anak 1:
  - Indeks dari gen yang disalin dari induk 2 ke anak 2: [4, 6, 7], gen: [2, 9, 8]
  - Indeks yang lain diisi dari induk 1 secara berurutan dengan melewatkan gen yang sudah muncul
  Anak 2:  [6, 1, 7, 5, 2, 3, 9, 8, 4]

Mutasi pada anak 1
Bilangan random: 0.5246 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 1: [1, 4, 6, 9, 7, 8, 2, 3, 5]

Mutasi pada anak 2
Bilangan random: 0.6820 > tingkat mutasi (0.1) --> mutasi tidak dilakukan
Anak 2: [6, 1, 7, 5, 2, 3, 9, 8, 4]
```

```
Generasi = 1
Populasi Baru:

Individu 1:    [3, 7, 9, 4, 2, 5, 8, 6, 1]
Individu 2:    [1, 3, 4, 7, 8, 2, 5, 9, 6]
Individu 3:    [5, 4, 3, 6, 1, 9, 7, 8, 2]
Individu 4:    [1, 5, 2, 7, 6, 4, 8, 3, 9]
Individu 5:    [3, 7, 1, 4, 8, 6, 5, 2, 9]
Individu 6:    [8, 4, 1, 6, 2, 5, 3, 9, 7]
Individu 7:    [8, 9, 1, 6, 5, 2, 3, 4, 7]
Individu 8:    [9, 6, 8, 1, 2, 5, 3, 7, 4]
Individu 9:    [1, 4, 6, 9, 7, 8, 2, 3, 5]
Individu 10:   [6, 1, 7, 5, 2, 3, 9, 8, 4]
```

```
Solusi, Nilai fungsi tujuan (jarak total), dan Nilai fitnes:
Nilai fitness = 1/Jarak total


Individu 1        Solusi: [0, 3, 7, 9, 4, 2, 5, 8, 6, 1, 0], Jarak total = 135, Nilai fitnes = 0.0074
Individu 2        Solusi: [0, 1, 3, 4, 7, 8, 2, 5, 9, 6, 0], Jarak total = 107, Nilai fitnes = 0.0093
Individu 3        Solusi: [0, 5, 4, 3, 6, 1, 9, 7, 8, 2, 0], Jarak total = 121, Nilai fitnes = 0.0083
Individu 4        Solusi: [0, 1, 5, 2, 7, 6, 4, 8, 3, 9, 0], Jarak total = 99,  Nilai fitnes = 0.0101
Individu 5        Solusi: [0, 3, 7, 1, 4, 8, 6, 5, 2, 9, 0], Jarak total = 105, Nilai fitnes = 0.0095
Individu 6        Solusi: [0, 8, 4, 1, 6, 2, 5, 3, 9, 7, 0], Jarak total = 89,  Nilai fitnes = 0.0112
Individu 7        Solusi: [0, 8, 9, 1, 6, 5, 2, 3, 4, 7, 0], Jarak total = 102, Nilai fitnes = 0.0098
Individu 8        Solusi: [0, 9, 6, 8, 1, 2, 5, 3, 7, 4, 0], Jarak total = 116, Nilai fitnes = 0.0086
Individu 9        Solusi: [0, 1, 4, 6, 9, 7, 8, 2, 3, 5, 0], Jarak total = 114, Nilai fitnes = 0.0088
Individu 10       Solusi: [0, 6, 1, 7, 5, 2, 3, 9, 8, 4, 0], Jarak total = 84,  Nilai fitnes = 0.0119

Solusi terbaik pada generasi ini:
Solusi: [0, 6, 1, 7, 5, 2, 3, 9, 8, 4, 0], Jarak total = 84,  Nilai fitnes = 0.0119
Nilai fitness tidak lebih baik dari solusi terbaik saat ini

Solusi terbaik saat ini --> tetap:
Solusi = [0, 9, 6, 8, 1, 7, 5, 2, 3, 4, 0], Jarak total = 83, Nilai fitness = 0.0120
```

Prosedur dilanjutkan hingga kriteria henti tercapai
(dalam hal ini hingga, misal, 100 generasi)

Solusi terbaik:
Solusi = [0, 9, 1, 3, 2, 6, 4, 8, 5, 7, 0], Jarak total = 56, Nilai fitness = 0.0179